

5.2 GPIO

- GPIO**：General Purpose I/O ports，通用输入/输出，ARM 芯片中称为输入输出端口。
- 说得简单点就是控制 GPIO 管脚电平的高低来控制外围器件，单片机差不多就是专门做这个事的。MCS-51 单片机的 P0-P3 口很简单，只需要在管脚相应的寄存储器写入 0 或 1 就行了。ARM 处理器要复杂一点，因为 ARM 大部分的管脚都不只有一个功能（输入/输出，多功能复用管脚），所以具体应用中要做个选择，也就是要配置相应的寄存储器，选定一个功能。只要把相应的寄存储器配置好后就和单片机一样的简单。
- 在实际应用中，不管是接 LCD、按键盒，控制 LED 流水灯等等，都离不开对 GPIO 的操作。可以说，对 GPIO 的操作是所有硬件操作的基础。
- S3C2440 的 GPIO**：130 个，分为 9 组，GPA~GPJ，这 9 组 GPIO 端口均为多功能端口，端口功能可以编程设置，若选定某个 GPIO 端口的功能，应在主程序运行之前编程设置对应的控制寄存器，从而选定所需 GPIO 端口的功能；如果某个 GPIO 引脚不用于特定功能，那么该引脚就可以设置为普通的输入输出引脚。

2. 端口 B 的 GPIO 引脚功能

- 端口 GPB 的 I/O 引脚共有 11 个，有 2 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作 DMA 请求信号（nXDREQ 等）和定时器输出信号（TOUT0-TOUT3）。

引脚名称	功能 1	功能 2	功能 3
GPB10	输入/输出	nXDREQ0	—
GPB9	输入/输出	nXDACK0	—
GPB8	输入/输出	nXDREQ1	—
GPB7	输入/输出	nXDACK1	—
GPB6	输入/输出	nXBREQ	—
GPB5	输入/输出	nXBACK	—
GPB4	输入/输出	TCLK0	—
GPB3	输入/输出	TOUT3	—
GPB2	输入/输出	TOUT2	—
GPB1	输入/输出	TOUT1	—
GPB0	输入/输出	TOUT0	—

5. 端口 E 的 GPIO 引脚功能

- 端口 GPE 的 I/O 引脚共有 16 个，有 3 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作 SPI 接口信号（GPE11~GPE13）、SD 卡接口信号（GPE5~GPE10）、IIC 接口信号和 IIS 接口信号。
- 功能 3：AC97 控制器接口信号（GPE0~GPE4）。

引脚名称	功能 1	功能 2	功能 3	引脚名称	功能 1	功能 2	功能 3
GPE15	输入/输出	IICSDA	—	GPE7	输入/输出	SDDAT0	—
GPE14	输入/输出	IIC_SCL	—	GPE6	输入/输出	SDCMD	—
GPE13	输入/输出	SPICLK0	—	GPE5	输入/输出	SDCLK	—
GPE12	输入/输出	SPIMOSI0	—	GPE4	输入/输出	I2SSDO	AC_SDATA_OUT
GPE11	输入/输出	SPIMISO0	—	GPE3	输入/输出	I2SSDI	AC_SDATA_IN
GPE10	输入/输出	SDDAT3	—	GPE2	输入/输出	CDCLK	AC_nRESET
GPE9	输入/输出	—	—	GPE1	输入/输出	—	—
GPE8	输入/输出	—	—	GPE0	输入/输出	—	—

5.2.1 端口功能

- S3C2440 有 130 个 GPIO 引脚，分布在 9 个 GPIO 端口：
- （1）端口 A：GPA，23 个引脚，无内部上拉电阻，主要用于输出、地址总线、Bank 选择、Nand flash 控制等。
- （2）端口 B：GPB，11 个引脚，主要用于输入输出、DMA 请求、定时器输出等。
- （3）端口 C：GPC，16 个引脚，主要用于输入输出、LCD 接口。
- （4）端口 D：GPD，16 个引脚，主要用于输入输出、LCD 接口、SPI 接口。
- （5）端口 E：GPE，16 个引脚，主要用于输入输出、SPI 接口、SD 卡接口、IIC 接口、IIS 接口、AC97 控制器接口。
- （6）端口 F：GPF，8 个引脚，主要用于输入输出、中断请求输入。
- （7）端口 G：GPG，16 个引脚，主要用于输入输出、中断请求输入、UART 接口、SPI 接口。
- （8）端口 H：GPH，11 个引脚，主要用于输入输出、时钟输出、UART 接口。
- （9）端口 J：GPJ，13 个引脚，主要用于输入输出、摄像头接口。
- 每个 GPIO 端口均是多功能的。

3. 端口 C 的 GPIO 引脚功能

- 端口 GPC 的 I/O 引脚共有 16 个，有 2 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作 LCD 控制器的接口信号（VFRAME、VCLK 等）和 LCD 像素点数据输出端口（VD0~VD7）。

引脚名称	功能 1	功能 2	功能 3	引脚名称	功能 1	功能 2	功能 3
GPC15	输入/输出	VD7	—	GPC7	输入/输出	LCD_LPCREVB	—
GPC14	输入/输出	VD6	—	GPC6	输入/输出	LCD_LPCREV	—
GPC13	输入/输出	VD5	—	GPC5	输入/输出	LCD_LPCOE	—
GPC12	输入/输出	VD4	—	GPC4	输入/输出	VM	—
GPC11	输入/输出	VD3	—	GPC3	输入/输出	VFRAME	—
GPC10	输入/输出	VD2	—	GPC2	输入/输出	VLINE	—
				GPC1	输入/输出	VCLK	—
				GPC0	输入/输出	LEND	—

6. 端口 F 的 GPIO 引脚功能

- 端口 GPF 的 I/O 引脚共有 8 个，有 2 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作中断源 EINT7~EINT0 的中断请求信号的输入。

引脚名称	功能 1	功能 2	功能 3
GPF7	输入/输出	EINT7	—
GPF6	输入/输出	EINT6	—
GPF5	输入/输出	EINT5	—
GPF4	输入/输出	EINT4	—
GPF3	输入/输出	EINT3	—
GPF2	输入/输出	EINT2	—
GPF1	输入/输出	EINT1	—
GPF0	输入/输出	EINT0	—

1. 端口 A 的 GPIO 引脚功能

- 端口 GPA 的 I/O 引脚共有 23 个，有 2 种功能：
- 功能 1：作为普通的输出（注意：只能作为输出）。
- 功能 2：主要用作地址信号、Bank 选择信号 nGCS、Nand flash 控制信号。

引脚名称	功能 1	功能 2	功能 3	引脚名称	功能 1	功能 2	功能 3
GPA22	输出	nFCE	—	GPA11	输出	ADDR26	—
GPA21	输出	nRSTOUT	—	GPA10	输出	ADDR25	—
GPA20	输出	nFRE	—	GPA9	输出	ADDR24	—
GPA19	输出	nFWE	—	GPA8	输出	ADDR23	—
GPA18	输出	ALE	—	GPA7	输出	ADDR22	—
GPA17	输出	CLE	—	GPA6	输出	ADDR21	—
GPA16	输出	nGCS5	—	GPA5	输出	ADDR20	—
GPA15	输出	nGCS4	—	GPA4	输出	ADDR19	—
GPA14	输出	nGCS3	—	GPA3	输出	ADDR18	—
GPA13	输出	nGCS2	—	GPA2	输出	ADDR17	—
GPA12	输出	nGCS1	—	GPA1	输出	ADDR16	—
				GPA0	输出	ADDR0	—

4. 端口 D 的 GPIO 引脚功能

- 端口 GPD 的 I/O 引脚共有 16 个，有 3 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作 LCD 像素点数据输出端口（VD8~VD23）。
- 功能 3：SPI 接口信号（GPD8、GPD9、GPD10）和 SPI 片选信号（nSS0、nSS1）。

引脚名称	功能 1	功能 2	功能 3	引脚名称	功能 1	功能 2	功能 3
GPD15	输入/输出	VD23	nSS0	GPD7	输入/输出	VD15	—
GPD14	输入/输出	VD22	nSS1	GPD6	输入/输出	VD14	—
GPD13	输入/输出	VD21	—	GPD5	输入/输出	VD13	—
GPD12	输入/输出	VD20	—	GPD4	输入/输出	VD12	—
GPD11	输入/输出	VD19	—	GPD3	输入/输出	VD11	—
GPD10	输入/输出	VD18	SPICLK1	GPD2	输入/输出	VD10	—
GPD9	输入/输出	VD17	SPIMOSI1				
GPD8	输入/输出	VD16	SPIMISO1				

7. 端口 G 的 GPIO 引脚功能

- 端口 GPG 的 I/O 引脚共有 16 个，有 3 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作中断源 EINT8~EINT23 的中断请求信号的输入。
- 功能 3：UART 接口信号、SPI 接口信号。

引脚名称	功能 1	功能 2	功能 3	引脚名称	功能 1	功能 2	功能 3
GPG15	输入/输出	EINT23	—	GPG7	输入/输出	EINT15	SPICLK1
GPG14	输入/输出	EINT22	—	GPG6	输入/输出	EINT14	SPIMOSI1
GPG13	输入/输出	EINT21	—	GPG5	输入/输出	EINT13	SPIMISO1
GPG12	输入/输出	EINT20	—	GPG4	输入/输出	EINT12	LCD_PWREN
GPG11	输入/输出	EINT19	TCLK1	GPG3	输入/输出	EINT11	nSS1
GPG10	输入/输出	EINT18	nCTS1	GPG2	输入/输出	EINT10	nSS0
GPG9	输入/输出	EINT17	nRTS1	GPG1	输入/输出	EINT9	—

8. 端口 H 的 GPIO 引脚功能

- 端口 GPH 的 I/O 引脚共有 11 个，有 3 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作时钟输出信号、UART 接口信号。
- 功能 3：UART 接口信号（CTS、RTS）。

引脚名称	功能 1	功能 2	功能 3
GPH10	输入/输出	CLKOUT1	—
GPH9	输入/输出	CLKOUT0	—
GPH8	输入/输出	UEXTCLK	—
GPH7	输入/输出	RXD2	nCTS1
GPH6	输入/输出	TXD2	nRTS1
GPH5	输入/输出	RXD1	—
GPH4	输入/输出	TXD1	—
GPH3	输入/输出	RXD0	—
GPH2	输入/输出	TXD0	—
GPH1	输入/输出	nRTS0	—
GPH0	输入/输出	nCTS0	—

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

10

1. 端口 A 的寄存器

- 端口 GPA 的寄存器包括：控制寄存器 GPACON、数据寄存器 GPADAT。
- 控制寄存器 GPACON：设置端口 A 中每个 GPIO 引脚的功能。
- GPACON 地址 = 0x56000000，复位值 = 0x7FFFFFFF，可读可写。
- 数据寄存器 GPADAT：存放端口 A 对应的数据。当端口 A 是输出功能时，S3C2440 通过 GPADAT 输出数据。
- GPADAT 地址 = 0x56000004，复位值 = 不确定，可读可写。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

13

GPADAT 数据寄存器格式

GPADAT	位	描述	初始值
GPA 22 : 0	[22:0]	存放端口 A 的数据。	—

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

16

9. 端口 J 的 GPIO 引脚功能

- 端口 GPJ 的 I/O 引脚共有 13 个，有 2 种功能：
- 功能 1：作为普通的输入/输出。
- 功能 2：主要用作摄像头接口信号。

引脚名称	功能 1	功能 2	功能 3
GPJ12	输入/输出	CAMRESET	—
GPJ11	输入/输出	CAMCLKOUT	—
GPJ10	输入/输出	CAMHREF	—
GPJ9	输入/输出	CAMVSYNC	—
GPJ8	输入/输出	CAMPCLK	—
GPJ7	输入/输出	CAMDATA7	—
GPJ6	输入/输出	CAMDATA6	—
GPJ5	输入/输出	CAMDATA5	—
GPJ4	输入/输出	CAMDATA4	—
GPJ3	输入/输出	CAMDATA3	—
GPJ2	输入/输出	CAMDATA2	—
GPJ1	输入/输出	CAMDATA1	—
GPJ0	输入/输出	CAMDATA0	—

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

11

GPACON 控制寄存器格式 1

GPACON	位	描述	初始值
GPA24	[24]	保留	—
GPA23	[23]	保留	—
GPA22	[22]	0 = 输出； 1 = nFCE。	1
GPA21	[21]	0 = 输出； 1 = nRSTOUT。	1
GPA20	[20]	0 = 输出； 1 = nFRE。	1
GPA19	[19]	0 = 输出； 1 = nFWE。	1
GPA18	[18]	0 = 输出； 1 = ALE。	1
GPA17	[17]	0 = 输出； 1 = CLE。	1
GPA16	[16]	0 = 输出； 1 = nGCS[5]。	1
GPA15	[15]	0 = 输出； 1 = nGCS[4]。	1
GPA14	[14]	0 = 输出； 1 = nGCS[3]。	1
GPA13	[13]	0 = 输出； 1 = nGCS[2]。	1
GPA12	[12]	0 = 输出； 1 = nGCS[1]。	1
GPA11	[11]	0 = 输出； 1 = ADDR26。	1
GPA10	[10]	0 = 输出； 1 = ADDR25。	1

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

14

2. 端口 B 的寄存器

- 端口 GPB 的寄存器包括：控制寄存器 GPBCON、数据寄存器 GPBDAT、上拉电阻寄存器 GPBUP。
- 控制寄存器 GPBCON：设置端口 B 中每个 GPIO 引脚的功能。
- GPBCON 地址 = 0x56000010，复位值 = 0x000000，可读可写。
- 数据寄存器 GPBDAT：存放端口 B 对应的数据。当该端口选择输入功能时，从该寄存器读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPBDAT 地址 = 0x56000014，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPBUP：确定端口 B 的 GPIO 引脚是否内部接上拉电阻。
- GPBUP 地址 = 0x56000018，复位值 = 0x000，可读可写。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

17

5.2.2 端口配置寄存器

- S3C2440 所有 I/O 引脚是功能复用的。因此，在具体使用 I/O 端口引脚时，应该通过编程设置端口控制寄存器，以决定使用每个 I/O 引脚的哪种功能。
- 另外，I/O 端口的状态（如输入/输出、数据线是否挂起），用户也需要通过编程设置控制寄存器来确定。
- S3C2440 有 9 个端口，每个端口一般对应 3 个寄存器（端口 A 只有 2 个）：控制寄存器（**CON）和数据寄存器（**DAT）、上拉电阻寄存器（**UP）。
- 比如端口 B 的寄存器表示为：GPBCON、GPBDAT、GPBUP。
- 控制寄存器功能：配置 I/O 引脚的功能，即一个 I/O 引脚选择哪个功能。控制寄存器有自己的寻址地址。
- 数据寄存器功能：保存输入、或者输出的数据。数据寄存器有自己的寻址地址。
- 上拉电阻寄存器：确定端口 B 的 GPIO 引脚是否内部接上拉电阻。上拉电阻寄存器有自己的寻址地址。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

12

GPACON 控制寄存器格式 2

GPACON	位	描述	初始值
GPA9	[9]	0 = 输出； 1 = ADDR24。	1
GPA8	[8]	0 = 输出； 1 = ADDR23。	1
GPA7	[7]	0 = 输出； 1 = ADDR22。	1
GPA6	[6]	0 = 输出； 1 = ADDR21。	1
GPA5	[5]	0 = 输出； 1 = ADDR20。	1
GPA4	[4]	0 = 输出； 1 = ADDR19。	1
GPA3	[3]	0 = 输出； 1 = ADDR18。	1
GPA2	[2]	0 = 输出； 1 = ADDR17。	1
GPA1	[1]	0 = 输出； 1 = ADDR16。	1
GPA0	[0]	0 = 输出； 1 = ADDR0。	1

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

15

GPBCON 控制寄存器格式

GPBCON	位	描述	初始值
GPB10	[21:20]	00 = 输入； 01 = 输出； 10 = nXDREQ0； 11 = 保留。	00
GPB9	[19:18]	00 = 输入； 01 = 输出； 10 = nXDACK0； 11 = 保留。	00
GPB8	[17:16]	00 = 输入； 01 = 输出； 10 = nXDREQ1； 11 = 保留。	00
GPB7	[15:14]	00 = 输入； 01 = 输出； 10 = nXDACK1； 11 = 保留。	00
GPB6	[13:12]	00 = 输入； 01 = 输出； 10 = nXBREQ； 11 = 保留。	00
GPB5	[11:10]	00 = 输入； 01 = 输出； 10 = nXBACK； 11 = 保留。	00
GPB4	[9:8]	00 = 输入； 01 = 输出； 10 = TCLK [0]； 11 = 保留。	00
GPB3	[7:6]	00 = 输入； 01 = 输出； 10 = TOUT3； 11 = 保留。	00
GPB2	[5:4]	00 = 输入； 01 = 输出； 10 = TOUT2； 11 = 保留。	00
GPB1	[3:2]	00 = 输入； 01 = 输出； 10 = TOUT1； 11 = 保留。	00
GPB0	[1:0]	00 = 输入； 01 = 输出； 10 = TOUT0； 11 = 保留。	00

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

18

GPBDAT 数据寄存器格式

GPBDAT	位	描述	初始值
GPB 10:0	[10:0]	存放端口 B 的数据。	—

GPCCON 控制寄存器格式

GPCCON	位	描述	初始值
GPC15	[31:30]	00= 输入；01= 输出；10=VD[7]；11= 保留。	00
GPC14	[29:28]	00= 输入；01= 输出；10=VD[6]；11= 保留。	00
GPC13	[27:26]	00= 输入；01= 输出；10=VD[5]；11= 保留。	00
GPC12	[25:24]	00= 输入；01= 输出；10=VD[4]；11= 保留。	00
GPC11	[23:22]	00= 输入；01= 输出；10=VD[3]；11= 保留。	00
GPC10	[21:20]	00= 输入；01= 输出；10=VD[2]；11= 保留。	00
GPC9	[19:18]	00= 输入；01= 输出；10=VD[1]；11= 保留。	00
GPC8	[17:16]	00= 输入；01= 输出；10=VD[0]；11= 保留。	00
GPC7	[15:14]	00= 输入；01= 输出；10=LCD_LPCREVB；11= 保留。	00
GPC6	[13:12]	00= 输入；01= 输出；10=LCD_LPCREV；11= 保留。	00
GPC5	[11:10]	00= 输入；01= 输出；10=LCD_LPCOE；11= 保留。	00
GPC4	[9:8]	00= 输入；01= 输出；10=VM；11= 保留。	00
GPC3	[7:6]	00= 输入；01= 输出；10=VFRAME；11= 保留。	00
GPC2	[5:4]	00= 输入；01= 输出；10=VLINE；11= 保留。	00
GPC1	[3:2]	00= 输入；01= 输出；10=VCLK；11= 保留。	00
GPC0	[1:0]	00= 输入；01= 输出；10=LEND；11= 保留。	00

4. 端口 D 的寄存器

- 端口 GPD 的寄存器包括：控制寄存器 GPDCON、数据寄存器 GPDDAT、上拉电阻寄存器 GPDUP。
- 控制寄存器 GPDCON：设置端口 D 中每个 GPIO 引脚的功能。
- GPDCON 地址 =0x56000030，复位值 =0x000000，可读可写。
- 数据寄存器 GPDDAT：存放端口 D 对应的数据。当该端口选择输入功能时，从该寄存器可读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPDDAT 地址 =0x56000034，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPDUP：确定端口 D 的 GPIO 引脚是否内部接上拉电阻。
- GPDUP 地址 =0x56000038，复位值 =0x0000，可读可写。

GPBUP 上拉电阻寄存器格式

GPBUP	位	描述	初始值
GPB 10:0	[10:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0x000

GPCDAT 数据寄存器格式

GPCDAT	位	描述	初始值
GPC 15:0	[15:0]	存放端口 C 的数据。	—

GPDCON 控制寄存器格式

GPDCON	位	描述	初始值
GPD15	[31:30]	00= 输入；01= 输出；10=VD23；11=nSS0。	00
GPD14	[29:28]	00= 输入；01= 输出；10=VD22；11= 保留。	00
GPD13	[27:26]	00= 输入；01= 输出；10=VD21；11= 保留。	00
GPD12	[25:24]	00= 输入；01= 输出；10=VD20；11= 保留。	00
GPD11	[23:22]	00= 输入；01= 输出；10=VD19；11= 保留。	00
GPD10	[21:20]	00= 输入；01= 输出；10=VD18；11= 保留。	00
GPD9	[19:18]	00= 输入；01= 输出；10=VD17；11= 保留。	00
GPD8	[17:16]	00= 输入；01= 输出；10=VD16；11= 保留。	00
GPD7	[15:14]	00= 输入；01= 输出；10=VD15；11= 保留。	00
GPD6	[13:12]	00= 输入；01= 输出；10=VD14；11= 保留。	00
GPD5	[11:10]	00= 输入；01= 输出；10=VD13；11= 保留。	00
GPD4	[9:8]	00= 输入；01= 输出；10=VD12；11= 保留。	00
GPD3	[7:6]	00= 输入；01= 输出；10=VD11；11= 保留。	00
GPD2	[5:4]	00= 输入；01= 输出；10=VD10；11= 保留。	00
GPD1	[3:2]	00= 输入；01= 输出；10=VD9；11= 保留。	00
GPD0	[1:0]	00= 输入；01= 输出；10=VD8；11= 保留。	00

3. 端口 C 的寄存器

- 端口 GPC 的寄存器包括：控制寄存器 GPCCON、数据寄存器 GPCDAT、上拉电阻寄存器 GPCUP。
- 控制寄存器 GPCCON：设置端口 C 中每个 GPIO 引脚的功能。
- GPCCON 地址 =0x56000020，复位值 =0x000000，可读可写。
- 数据寄存器 GPCDAT：存放端口 C 对应的数据。当该端口选择输入功能时，从该寄存器可读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPCDAT 地址 =0x56000024，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPCUP：确定端口 C 的 GPIO 引脚是否内部接上拉电阻。
- GPCUP 地址 =0x56000028，复位值 =0x0000，可读可写。

GPCUP 上拉电阻寄存器格式

GPCUP	位	描述	初始值
GPC 15:0	[15:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0x000

GPDDAT 数据寄存器格式

GPDDAT	位	描述	初始值
GPD 15:0	[15:0]	存放端口 D 的数据。	—

GPDUP 上拉电阻寄存器格式

GPDUP	位	描述	初始值
GPD 15:0	[15:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0x000

GPECON 控制寄存器格式 2

GPECON	位	描述	初始值
GPE7	[15:14]	00= 输入；01= 输出；10= SDDAT0；11= 保留。	00
GPE6	[13:12]	00= 输入；01= 输出；10= SDCMD；11= 保留。	00
GPE5	[11:10]	00= 输入；01= 输出；10= SDCLK；11= 保留。	00
GPE4	[9:8]	00= 输入；01= 输出；10= I2SDO； 11= AC_SDATA_OUT。	00
GPE3	[7:6]	00= 输入；01= 输出；10= I2SDI；11= AC_SDATA_IN。	00
GPE2	[5:4]	00= 输入；01= 输出；10= CDCLK；11= AC_nRESET。	00
GPE1	[3:2]	00= 输入；01= 输出；10= I2SSCLK；11= AC_BIT_CLK。	00
GPE0	[1:0]	00= 输入；01= 输出；10= I2SLRCK；11= AC_SYNC。	00

6. 端口 F 的寄存器

- 端口 GPF 的寄存器包括：控制寄存器 GPFCON、数据寄存器 GPFDAT、上拉电阻寄存器 GPFUP。
- 控制寄存器 GPFCON：设置端口 F 中每个 GPIO 引脚的功能。
- GPFCON 地址 = 0x56000050，复位值 = 0x000000，可读可写。
- 数据寄存器 GPFDAT：存放端口 F 对应的数据。当该端口选择输入功能时，从该寄存器可读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPFDAT 地址 = 0x56000054，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPFUP：确定端口 F 的 GPIO 引脚是否内部接上拉电阻。
- GPFUP 地址 = 0x56000058，复位值 = 0x00，可读可写。

5. 端口 E 的寄存器

- 端口 GPE 的寄存器包括：控制寄存器 GPECON、数据寄存器 GPEDAT、上拉电阻寄存器 GPEUP。
- 控制寄存器 GPECON：设置端口 E 中每个 GPIO 引脚的功能。
- GPECON 地址 = 0x56000040，复位值 = 0x000000，可读可写。
- 数据寄存器 GPEDAT：存放端口 E 对应的数据。当该端口选择输入功能时，从该寄存器可读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPEDAT 地址 = 0x56000044，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPEUP：确定端口 E 的 GPIO 引脚是否内部接上拉电阻。
- GPEUP 地址 = 0x56000048，复位值 = 0x0000，可读可写。

GPEDAT 数据寄存器格式

GPEDAT	位	描述	初始值
GPE 15:0	[15:0]	存放端口 E 的数据。	—

GPFCON 控制寄存器格式

GPFCON	位	描述	初始值
GPF7	[15:14]	00= 输入；01= 输出；10= EINT[7]；11= 保留。	00
GPF6	[13:12]	00= 输入；01= 输出；10= EINT[6]；11= 保留。	00
GPF5	[11:10]	00= 输入；01= 输出；10= EINT[5]；11= 保留。	00
GPF4	[9:8]	00= 输入；01= 输出；10= EINT[4]；11= 保留。	00
GPF3	[7:6]	00= 输入；01= 输出；10= EINT[3]；11= 保留。	00
GPF2	[5:4]	00= 输入；01= 输出；10= EINT[2]；11= 保留。	00
GPF1	[3:2]	00= 输入；01= 输出；10= EINT[1]；11= 保留。	00
GPF0	[1:0]	00= 输入；01= 输出；10= EINT[0]；11= 保留。	00

GPECON 控制寄存器格式 1

GPECON	位	描述	初始值
GPE15	[31:30]	00= 输入；01= 输出；10= IICSDA；11= 保留。 注：这个引脚是无上拉漏极开路（MOS 管）。	00
GPE14	[29:28]	00= 输入；01= 输出；10= IICSCS；11= 保留。 注：这个引脚是无上拉漏极开路（MOS 管）。	00
GPE13	[27:26]	00= 输入；01= 输出；10= SPICLK0；11= 保留。	00
GPE12	[25:24]	00= 输入；01= 输出；10= SPIMOSI0；11= 保留。	00
GPE11	[23:22]	00= 输入；01= 输出；10= SPIMISO0；11= 保留。	00
GPE10	[21:20]	00= 输入；01= 输出；10= SDDAT3；11= 保留。	00
GPE9	[19:18]	00= 输入；01= 输出；10= SDDAT2；11= 保留。	00
GPE8	[17:16]	00= 输入；01= 输出；10= SDDAT1；11= 保留。	00

GPEUP 上拉电阻寄存器格式

GPEUP	位	描述	初始值
GPE 15:0	[15:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0x000

GPFDAT 数据寄存器格式

GPFDAT	位	描述	初始值
GPF 7:0	[7:0]	存放端口 F 的数据。	—

GPFUP 上拉电阻寄存器格式

GPFUP	位	描述	初始值
GPF 7:0	[7:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0x000

GPGDAT 数据寄存器格式

GPGDAT	位	描述	初始值
GPG 15:0	[15:0]	存放端口 G 的数据。	—

GPHCON 控制寄存器格式

GPHCON	位	描述	初始值
GPH10	[21:20]	00= 输入；01= 输出；10= CLKOUT1；11= 保留。	00
GPH9	[19:18]	00= 输入；01= 输出；10= CLKOUT0；11= 保留。	00
GPH8	[17:16]	00= 输入；01= 输出；10= UEXTCLK；11= 保留。	00
GPH7	[15:14]	00= 输入；01= 输出；10= RXD[2]；11= nCTS1。	00
GPH6	[13:12]	00= 输入；01= 输出；10= TXD[2]；11= nRTS1。	00
GPH5	[11:10]	00= 输入；01= 输出；10= RXD[1]；11= 保留。	00
GPH4	[9:8]	00= 输入；01= 输出；10= TXD[1]；11= 保留。	00
GPH3	[7:6]	00= 输入；01= 输出；10= RXD[0]；11= 保留。	00
GPH2	[5:4]	00= 输入；01= 输出；10= TXD[0]；11= 保留。	00
GPH1	[3:2]	00= 输入；01= 输出；10= nRTS0；11= 保留。	00
GPH0	[1:0]	00= 输入；01= 输出；10= nCTS0；11= 保留。	00

7. 端口 G 的寄存器

- 端口 GPG 的寄存器包括：控制寄存器 GPGCON、数据寄存器 GPGDAT、上拉电阻寄存器 GPGUP。
- 控制寄存器 GPGCON：设置端口 G 中每个 GPIO 引脚的功能。
- GPGCON 地址 = 0x56000060，复位值 = 0x00000000，可读可写。
- 数据寄存器 GPGDAT：存放端口 G 对应的数据。当该端口选择输入功能时，从该寄存器可读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPGDAT 地址 = 0x56000064，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPGUP：确定端口 G 的 GPIO 引脚是否内部接上拉电阻。
- GPGUP 地址 = 0x56000068，复位值 = 0xF800，可读可写。

GPGUP 上拉电阻寄存器格式

GPGUP	位	描述	初始值
GPG 15:0	[15:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0xF800

GPHDAT 数据寄存器格式

GPHDAT	位	描述	初始值
GPH 10:0	[10:0]	存放端口 H 的数据。	—

GPGCON 控制寄存器格式

GPGCON	位	描述	初始值
GPG15	[31:30]	00= 输入；01= 输出；10= EINT[23]；11= 保留。	00
GPG14	[29:28]	00= 输入；01= 输出；10= EINT[22]；11= 保留。	00
GPG13	[27:26]	00= 输入；01= 输出；10= EINT[21]；11= 保留。	00
GPG12	[25:24]	00= 输入；01= 输出；10= EINT[20]；11= 保留。	00
GPG11	[23:22]	00= 输入；01= 输出；10= EINT[19]；11= TCLK[1]。	00
GPG10	[21:20]	00= 输入；01= 输出；10= EINT[18]；11= nCTS1。	00
GPG9	[19:18]	00= 输入；01= 输出；10= EINT[17]；11= nRTS1。	00
GPG8	[17:16]	00= 输入；01= 输出；10= EINT[16]；11= 保留。	00
GPG7	[15:14]	00= 输入；01= 输出；10= EINT[15]；11= SPICLK1。	00
GPG6	[13:12]	00= 输入；01= 输出；10= EINT[14]；11= SPIMOS1。	00
GPG5	[11:10]	00= 输入；01= 输出；10= EINT[13]；11= SPIMISO1。	00
GPG4	[9:8]	00= 输入；01= 输出；10= EINT[12]；11= LCD_PWRDN。	00
GPG3	[7:6]	00= 输入；01= 输出；10= EINT[11]；11= nSS1。	00
GPG2	[5:4]	00= 输入；01= 输出；10= EINT[10]；11= nSS0。	00
GPG1	[3:2]	00= 输入；01= 输出；10= EINT[9]；11= 保留。	00
GPG0	[1:0]	00= 输入；01= 输出；10= EINT[8]；11= 保留。	00

8. 端口 H 的寄存器

- 端口 GPH 的寄存器包括：控制寄存器 GPHCON、数据寄存器 GPHDAT、上拉电阻寄存器 GPHUP。
- 控制寄存器 GPHCON：设置端口 H 中每个 GPIO 引脚的功能。
- GPHCON 地址 = 0x56000070，复位值 = 0x00000000，可读可写。
- 数据寄存器 GPHDAT：存放端口 H 对应的数据。当该端口选择输入功能时，从该寄存器可读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPHDAT 地址 = 0x56000074，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPHUP：确定端口 H 的 GPIO 引脚是否内部接上拉电阻。
- GPHUP 地址 = 0x56000078，复位值 = 0x000，可读可写。

GPHUP 上拉电阻寄存器格式

GPHUP	位	描述	初始值
GPH 10:0	[10:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0x000

9. 端口 J 的寄存器

- 端口 GPJ 的寄存器包括：控制寄存器 GPJCON、数据寄存器 GPJDAT、上拉电阻寄存器 GPJUP。
- 控制寄存器 GPJCON：设置端口 J 中每个 GPIO 引脚的功能。
- GPJCON 地址 =0x560000D0，复位值 = 0x00000000，可读可写。
- 数据寄存器 GPJDAT：存放端口 J 对应的数据。当该端口选择输入功能时，从该寄存器可读取与端口连接的外部数据信息。当该端口选择输出功能时，向该寄存器写入的数据将通过端口输出。
- GPJDAT 地址 =0x560000D4，复位值 = 不确定，可读可写。
- 上拉电阻寄存器 GPJUP：确定端口 J 的 GPIO 引脚是否内部接上拉电阻。
- GPJUP 地址 =0x560000D8，复位值 =0x000，可读可写。

GPJUP 上拉电阻寄存器格式

GPJUP	位	描述	初始值
GPJ 12:0	[12:0]	1= 对应的 GPIO 引脚上拉电阻不使能； 0= 对应的 GPIO 引脚上拉电阻使能。	0x000

MISCCR 寄存器格式

MISCCR	位	描述	初始值
SEL_SUSPND1	[13]	USB 端口 1 的模式选择端。 0= 正常模式；1= 悬挂模式。	0
SEL_SUSPND0	[12]	USB 端口 0 的模式选择端。 0= 正常模式；1= 悬挂模式。	0
CLKSEL1 (Note)	[10:8]	CLKOUT1 的输出信号源。 000=MPLL CLK；001=UPLL CLK； 010=FCLK；011=HCLK；100=PCLK；101=DCLK1； 11x= 保留。	000
Reserved	[7]	保留。	0
CLKSEL0 (Note)	[6:4]	CLKOUT0 的输出信号源。 000=MPLL CLK；001=UPLL CLK； 010=FCLK；011=HCLK；100=PCLK；101=DCLK1； 11x= 保留。	010
SEL_USBPAD	[3]	USB 端口的模式选择端。 0= 正常模式；1= 悬挂模式（USB 数据线空闲 3ms 后，进入悬挂模式，为节省功耗）。	0
Reserved	[2]	保留。	0
SPUCR1	[1]	数据线 DATA15-DATA0 的上拉电阻使能端。0= 使能；1= 不使能。	0
SPUCR0	[0]	数据线 DATA31-DATA16 的上拉电阻使能端。 0= 使能；1= 不使能。	0

GPJCON 控制寄存器格式

GPJCON	位	描述	初始值
GPJ12	[25:24]	00= 输入；01= 输出；10= CAMRESET；11= 保留。	00
GPJ11	[23:22]	00= 输入；01= 输出；10= CAMCLKOUT；11= 保留。	00
GPJ10	[21:20]	00= 输入；01= 输出；10= CAMHREF；11= 保留。	00
GPJ9	[19:18]	00= 输入；01= 输出；10= CAMVSYNC；11= 保留。	00
GPJ8	[17:16]	00= 输入；01= 输出；10= CAMCLK；11= 保留。	00
GPJ7	[15:14]	00= 输入；01= 输出；10= CAMDATA[7]；11= 保留。	00
GPJ6	[13:12]	00= 输入；01= 输出；10= CAMDATA[6]；11= 保留。	00
GPJ5	[11:10]	00= 输入；01= 输出；10= CAMDATA[5]；11= 保留。	00
GPJ4	[9:8]	00= 输入；01= 输出；10= CAMDATA[4]；11= 保留。	00
GPJ3	[7:6]	00= 输入；01= 输出；10= CAMDATA[3]；11= 保留。	00
GPJ2	[5:4]	00= 输入；01= 输出；10= CAMDATA[2]；11= 保留。	00
GPJ1	[3:2]	00= 输入；01= 输出；10= CAMDATA[1]；11= 保留。	00
GPJ0	[1:0]	00= 输入；01= 输出；10= CAMDATA[0]；11= 保留。	00

5.2.3 端口其他寄存器

包括：

- 多控制寄存器 MISCCR。
- DCLK 控制寄存器 DCLKCON。
- 外部中断控制寄存器 EXTINT0、EXTINT1、EXTINT2。
- 外部中断过滤寄存器 EINTFLT0、EINTFLT1、EINTFLT2、EINTFLT3。
- 外部中断屏蔽寄存器 MINTMASK。
- 外部中断未决寄存器 EINTPEND。
- 通用状态寄存器 GSTATUS0、GSTATUS1、GSTATUS2、GSTATUS3、GSTATUS4。
- 驱动能力控制寄存器 DSC0、DSC1。
- 内存休眠控制寄存器 MSLCON。

2. DCLK 控制寄存器 DCLKCON

- DCLK 控制寄存器 DCLKCON：对外部源时钟 DCLK0、DCLK1 进行控制。
- DCLKCON 地址 =0x56000084，复位值 =0x00000000，可读可写。

DCLKCON	位	描述	初始值
DCLKICMP	[27:24]	确定 DCLK1 的占空比。 例如：DCLKICMP 的值为 n，那么 DCLK1 信号的低电平宽度为：n+1；DCLK1 信号的高电平宽度为：((DCLK1DIV + 1) - (n + 1))。	0000
DCLK1DIV	[23:20]	DCLK1 的分频系数。 DCLK1 信号的频率 = 源时钟频率 / (DCLK1DIV + 1)。	0000
Reserved	[19:18]	保留。	00
DCLK1SelCK	[17]	DCLK1 的源时钟选择端。 1=UCLK(USB)；0=PCLK。	0

GPJDAT 数据寄存器格式

GPJDAT	位	描述	初始值
GPJ 12:0	[12:0]	存放端口 J 的数据。	—

1. 多控制寄存器 MISCCR

- 多控制寄存器 MISCCR：对 USB 主机、USB 设备、CLKOUT 时钟选择、电池模式进行控制。
- MISCCR 地址 =0x56000080，复位值 =0x10330，可读可写。

MISCCR	位	描述	初始值
Reserved	[24:23]	保留。	00
BATT_FUNC	[22:20]	电池故障功能选择。 0XX: 当 In nBATT_FLT=0，系统复位，为防止电池启动故障状态。 10X: 休眠模式，当 nBATT_FLT=0 时，系统会唤醒；在正常模式时，nBATT_FLT = 0, 电池故障中断将发生。 110: 休眠模式，nBATT_FLT = 0 系统会忽视所有的觉醒的事件；正常模式，nBATT_FLT 信号不能影响系统。 111: nBATT_FLT 功能无效。	000
OFFREFRESH	[19]	0= 自刷新不使能；1= 自刷新使能。	0
nEN_SCLK1	[18]	SCLK1 信号输出使能。 0=SCLK1 等于 SCLK；1= SCLK1 为低电平。	0
nEN_SCLK0	[17]	SCLK0 信号输出使能。 0=SCLK0 等于 SCLK；1= SCLK0 低电平。	0
nRSTCON	[16]	用于 nRSTCON 信号软件控制。 0=nRSTCON 信号为 0；1=nRSTCON 信号为 1。	1
Reserved	[15:14]	保留。	00

DCLKCON 寄存器格式

DCLKCON	位	描述	初始值
DCLK1EN	[16]	DCLK1 的使能位。 1= DCLK1 使能；0= DCLK1 不使能。	0
Reserved	[15:12]	保留。	0000
DCLK0CMP	[11:8]	确定 DCLK0 的占空比。	0000
DCLK0DIV	[7:4]	DCLK0 的分频系数。	0000
Reserved	[3: 2]	保留。	00
DCLK0SelCK	[1]	DCLK0 的源时钟选择端。 1=UCLK(USB)；0=PCLK。	0
DCLK0EN	[0]	DCLK0 的源时钟选择端。 1=UCLK(USB)；0=PCLK。	0

位 [0:11] 控制 DCLK0 信号；位 [16:27] 控制 DCLK01 信号

3. 外部中断控制寄存器 EXTINTn

- 外部中断控制寄存器共有 3 个，分别为 EXTINT0、EXTINT1、EXTINT2。
- 功能：对 24 个外部中断请求信号（EINT0~EINT23）的有效触发方式进行选择。
- EXTINT0 地址 =0x56000088，复位值 =0x00000000，可读可写。
- EXTINT1 地址 =0x5600008c，复位值 =0x00000000，可读可写。
- EXTINT2 地址 =0x56000090，复位值 =0x00000000，可读可写。

EXTINT1 寄存器格式 1

EXTINT	位	描述	初始值
FLTEN15	[31]	确定 EINT15 的滤波器使能。 1= 使能；0= 不使能。	
EINT15	[30:28]	确定 EINT15 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN14	[27]	确定 EINT14 的滤波器使能。 1= 使能；0= 不使能。	
EINT14	[26:24]	确定 EINT14 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN13	[23]	确定 EINT13 的滤波器使能。 1= 使能；0= 不使能。	
EINT13	[22:20]	确定 EINT13 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN12	[19]	确定 EINT12 的滤波器使能。 1= 使能；0= 不使能。	
EINT12	[18:16]	确定 EINT12 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000

EXTINT2 寄存器格式 2

EINT20	[18:16]	确定 EINT20 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN19	[15]	确定 EINT19 的滤波器使能。 1= 使能；0= 不使能。	
EINT19	[14:12]	确定 EINT19 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN18	[11]	确定 EINT18 的滤波器使能。 1= 使能；0= 不使能。	
EINT18	[10:8]	确定 EINT18 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN17	[7]	确定 EINT17 的滤波器使能。 1= 使能；0= 不使能。	
EINT17	[6:4]	确定 EINT17 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN16	[3]	确定 EINT16 的滤波器使能。 1= 使能；0= 不使能。	
EINT16	[2:0]	确定 EINT16 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000

EXTINT0 寄存器格式 1

EXTINT	位	描述	初始值
EINT7	[30:28]	确定 EINT7 信号有效方式。 000= 低电平；001= 高电平；01×= 下降沿； 10×= 上升沿；11×= 边沿。	000
EINT6	[26:24]	确定 EINT6 信号有效方式。 000= 低电平；001= 高电平；01×= 下降沿； 10×= 上升沿；11×= 边沿。	000
EINT5	[22:20]	确定 EINT5 信号有效方式。 000= 低电平；001= 高电平；01×= 下降沿； 10×= 上升沿；11×= 边沿。	000
EINT4	[18:16]	确定 EINT4 信号有效方式。 000= 低电平；001= 高电平；01×= 下降沿； 10×= 上升沿；11×= 边沿。	000

EXTINT1 寄存器格式 2

FLTEN11	[15]	确定 EINT11 的滤波器使能。 1= 使能；0= 不使能。	
EINT11	[14:12]	确定 EINT11 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN10	[11]	确定 EINT10 的滤波器使能。 1= 使能；0= 不使能。	
EINT10	[10:8]	确定 EINT10 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN9	[7]	确定 EINT9 的滤波器使能。 1= 使能；0= 不使能。	
EINT9	[6:4]	确定 EINT9 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN8	[3]	确定 EINT8 的滤波器使能。 1= 使能；0= 不使能。	
EINT8	[2:0]	确定 EINT8 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000

4. 外部中断过滤寄存器 EINTFLTn

- 为了使 S3C2440 芯片有效检测到中断请求信号的电平，外部中断引脚的有效逻辑电平至少得保持 40ns。
- 外部中断过滤寄存器共有 4 个：分别为 EINTFLT0、EINTFLT1、EINTFLT2、EINTFLT3，它们均是可读/写的。
- EINTFLT0、EINTFLT1 保留，其地址分别为 0x56000094、0x56000098。
- EINTFLT2、EINTFLT3：分别用于对 8 个外部中断请求信号（EINT23-16）的滤波器宽度进行设置。其地址分别为 0x5600009C、0x4C6000A0。
- 4 个外部中断过滤寄存器复位值为：0x00000000。

EXTINT0 寄存器格式 2

EXTINT	位	描述	初始值
EINT3	[14:12]	确定 EINT3 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
EINT2	[10:8]	确定 EINT2 信号有效方式。 000= 低电平；001= 高电平；01×= 下降沿； 10×= 上升沿；11×= 边沿。	000
EINT1	[6:4]	确定 EINT1 信号有效方式。 000= 低电平；001= 高电平；01×= 下降沿； 10×= 上升沿；11×= 边沿。	000
EINT0	[2:0]	确定 EINT0 信号有效方式。 000= 低电平；001= 高电平；01×= 下降沿； 10×= 上升沿；11×= 边沿。	000

EXTINT2 寄存器格式 1

EXTINT	位	描述	初始值
FLTEN23	[31]	确定 EINT23 的滤波器使能。 1= 使能；0= 不使能。	
EINT23	[30:28]	确定 EINT23 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN22	[27]	确定 EINT22 的滤波器使能。 1= 使能；0= 不使能。	
EINT22	[26:24]	确定 EINT22 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN21	[23]	确定 EINT21 的滤波器使能。 1= 使能；0= 不使能。	
EINT21	[22:20]	确定 EINT21 信号有效方式。 000= 低电平；001= 高电平； 01×= 下降沿；10×= 上升沿；11×= 边沿。	000
FLTEN20	[19]	确定 EINT20 的滤波器使能。 1= 使能；0= 不使能。	

EINTFLT2 寄存器格式

EINTFLT	位	描述	初始值
FLTCLK19	[31]	EINT19 的滤波器时钟选择端。 0=PCLK；1=EXTCLK/OSC_CLK。	0
EINTFLT19	[30:24]	确定 EINT19 的滤波器宽度	0000000
FLTCLK18	[23]	EINT18 的滤波器时钟选择端。 0=PCLK；1=EXTCLK/OSC_CLK。	0
EINTFLT18	[22:16]	确定 EINT18 的滤波器宽度	0000000
FLTCLK17	[15]	EINT17 的滤波器时钟选择端。 0=PCLK；1=EXTCLK/OSC_CLK。	0
EINTFLT17	[14:8]	确定 EINT17 的滤波器宽度	0000000
FLTCLK16	[7]	EINT16 的滤波器时钟选择端。 0=PCLK；1=EXTCLK/OSC_CLK。	0
EINTFLT16	[6:0]	确定 EINT16 的滤波器宽度	0000000

EINTFLT3 寄存器格式

EINTFLT	位	描述	初始值
FLTCLK23	[31]	EINT23 的滤波器时钟选择端。 0=PCLK ; 1=EXTCLK/OSC_CLK。	0
EINTFLT23	[30:24]	确定 EINT23 的滤波器宽度	0000000
FLTCLK22	[23]	EINT22 的滤波器时钟选择端。 0=PCLK ; 1=EXTCLK/OSC_CLK。	0
EINTFLT22	[22:16]	确定 EINT22 的滤波器宽度	0000000
FLTCLK21	[15]	EINT21 的滤波器时钟选择端。 0=PCLK ; 1=EXTCLK/OSC_CLK。	0
EINTFLT21	[14:8]	确定 EINT21 的滤波器宽度	0000000
FLTCLK20	[7]	EINT20 的滤波器时钟选择端。 0=PCLK ; 1=EXTCLK/OSC_CLK。	0
EINTFLT20	[6:0]	确定 EINT20 的滤波器宽度	0000000

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 64

EINTPEND 寄存器格式

EINT13	[13]	EINT13 的中断未决位。0 = 不请求；1 = 请求。	0
EINT12	[12]	EINT12 的中断未决位。0 = 不请求；1 = 请求。	0
EINT11	[11]	EINT11 的中断未决位。0 = 不请求；1 = 请求。	0
EINT10	[10]	EINT10 的中断未决位。0 = 不请求；1 = 请求。	0
EINT9	[9]	EINT9 的中断未决位。0 = 不请求；1 = 请求。	0
EINT8	[8]	EINT8 的中断未决位。0 = 不请求；1 = 请求。	0
EINT7	[7]	EINT7 的中断未决位。0 = 不请求；1 = 请求。	0
EINT6	[6]	EINT6 的中断未决位。0 = 不请求；1 = 请求。	0
EINT5	[5]	EINT5 的中断未决位。0 = 不请求；1 = 请求。	0
EINT4	[4]	EINT4 的中断未决位。0 = 不请求；1 = 请求。	0
Reserved	[3 : 0]	保留	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 67

GSTATUS1 寄存器格式

GSTATUS	位	描述
CHIP ID	[0]	ID 寄存器复位值为 0x32440001。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 70

5. 外部中断屏蔽寄存器 EINTMSK

- 外部中断屏蔽寄存器 EINTMSK：用于屏蔽 20 个外部中断（EINT4~EINT23）。
- EINTMSK 地址 = 0x560000A4，复位值 = 0x00FFFFFF0，可读可写。

EINTMSK	位	描述	初始值
EINT4~EINT23	[23:4]	EINT4~EINT23 的屏蔽位 0 = 不屏蔽；1 = 屏蔽	[23:4] 位都是 1
Reserved	[3:0]	保留	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 65

7. 通用状态寄存器 GSTATUSn

- 通用状态寄存器包括 GSTATUS0、GSTATUS1、GSTATUS2、GSTATUS3、GSTATUS4 五个状态寄存器。
- GSTATUS0 寄存器：反映外部引脚状态，地址 = 0x560000AC，复位值 = 未确定，只读。
- GSTATUS1 寄存器：反映芯片编号情况，地址 = 0x560000B0，复位值 = 0x32440001，只读。
- GSTATUS2 寄存器：反映复位状态，地址 = 0x560000B4，复位值 = 0x1，可读可写。
- GSTATUS3 寄存器：当上电复位时，该寄存器被清空，否则保存数据。地址 = 0x560000B8，复位值 = 0x0，可读可写。
- GSTATUS4 寄存器：当上电复位时，该寄存器被清空，否则保存数据。地址 = 0x560000BC，复位值 = 0x0，可读可写。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 68

GSTATUS2 寄存器格式

GSTATUS	位	描述
Reserved	[3]	保留。
WDTRST	[2]	置 1 时看门狗复位。
SLEEPST	[1]	置 1 时省电模式复位。
PWRST	[0]	置 1 时上电复位。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 71

6. 外部中断未决寄存器 EINTPND

- 外部中断未决寄存器 EINTPND：保存 20 个外部中断（EINT4~EINT23）的中断请求状态。1 = 有中断请求。
- EINTPND 其地址 = 0x560000A8，复位值 = 0x00，可读可写。

EINTPND	位	描述	初始值
EINT23	[23]	EINT23 的中断未决位。0 = 不请求；1 = 请求。	0
EINT22	[22]	EINT22 的中断未决位。0 = 不请求；1 = 请求。	0
EINT21	[21]	EINT21 的中断未决位。0 = 不请求；1 = 请求。	0
EINT20	[20]	EINT20 的中断未决位。0 = 不请求；1 = 请求。	0
EINT19	[19]	EINT19 的中断未决位。0 = 不请求；1 = 请求。	0
EINT18	[18]	EINT18 的中断未决位。0 = 不请求；1 = 请求。	0
EINT17	[17]	EINT17 的中断未决位。0 = 不请求；1 = 请求。	0
EINT16	[16]	EINT16 的中断未决位。0 = 不请求；1 = 请求。	0
EINT15	[15]	EINT15 的中断未决位。0 = 不请求；1 = 请求。	0
EINT14	[14]	EINT14 的中断未决位。0 = 不请求；1 = 请求。	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 66

GSTATUS0 寄存器格式

GSTATUS	位	描述
nWAIT	[3]	nWAIT 引脚状态。
NCON	[2]	NCON 引脚状态。
RnB	[1]	RnB 引脚状态。
BATT_FLT	[0]	BATT_FLT 引脚状态。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 69

GSTATUS3 寄存器格式

GSTATUS	位	描述
inform	[31:0]	当上电复位时，该寄存器被清空，否则保存数据。 (可被复位信号 nRESET 或看门狗定时器清 0)

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 72

GSTATUS4 寄存器格式

GSTATUS	位	描述
inform	[31:0]	当上电复位时, 该寄存器被清空, 否则保存数据。 (可被复位信号 nRESET 或看门狗定时器清 0)

DSC1 寄存器格式 1

DSC	位	描述	初始值
DSC_SCK1	[29:28]	SCLK1 驱动线能力。 00=12mA ; 10=10mA ; 01=8mA ; 11=6mA 。	0
DSC_SCK0	[27:26]	SCLK0 驱动线能力。 00=12mA ; 10=10mA ; 01=8mA ; 11=6mA ;	0
DSC_SCKE	[25:24]	SCKE 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_SDR	[23:22]	nSRAS/nSCAS 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_NFC	[21:20]	Nand flash 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_BE	[19:18]	nBE[3:0] 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_WOE	[17:16]	nWE/nOE 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_CS7	[15:14]	nGCS7 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00

MSLCON 寄存器格式

MSLCON	位	描述	初始值
PSC_DATA	[11]	在休眠模式下, DATA[31:0] 引脚状态。0 = 高阻 ; 1 = 输出“0”。	0
PSC_WAIT	[10]	在休眠模式下, nWAIT 引脚状态。0 = 输入 ; 1 = 输出“0”。	0
PSC_RnB	[9]	在休眠模式下, RnB 引脚状态。0 = 输入 ; 1 = 输出“0”。	0
PSC_NF	[8]	在休眠模式下, NAND Flash 引脚状态。 0 = 闲置 ; (即 nFCE,nFRE,nFWE,ALE,CLE=11100) 1 = 高阻状态。	0
PSC_SDR	[7]	在休眠模式下, nSRAS, nSCAS 引脚状态。 0 = 闲置 (“1”) ; 1 = 高阻状态。	0
PSC_DQM	[6]	在休眠模式下, DQM[3:0]/nWE[3:0] 引脚状态。 0 = 闲置 (“1”) ; 1 = 高阻状态。	0
PSC_OE	[5]	在休眠模式下, nOE 引脚状态。0 = 闲置 (“1”) ; 1 = 高阻状态。	0
PSC_WE	[4]	在休眠模式下, nWE 引脚状态。0 = 闲置 (“1”) ; 1 = 高阻状态。	0
PSC_GCS0	[3]	在休眠模式下, nGCS[0] 引脚状态。0 = 闲置 (“1”) ; 1 = 高阻状态。	0
PSC_GCS51	[2]	在休眠模式下, nGCS[5:1] 引脚状态。0 = 闲置 (“1”) ; 1 = 高阻状态。	0
PSC_GCS6	[1]	在休眠模式下, nGCS[6] 引脚状态。0 = 闲置 (“1”) ; 1 = 高阻状态。	0
PSC_GCS7	[0]	在休眠模式下, nGCS[7] 引脚状态。0 = 闲置 (“1”) ; 1 = 高阻状态。	0

8. 驱动能力控制寄存器 DSCn

- 驱动能力控制寄存器包括 DSC0、DSC1。
- 驱动能力控制寄存器 **DSC0** 功能：配置地址线、数据线的 I/O 驱动能力。
- DSC0 地址 = 0x560000C4，复位值 = 0x00000000，可读可写。

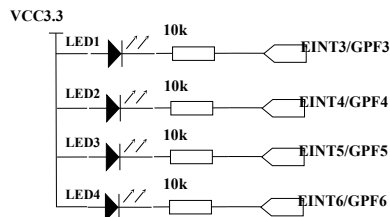
- 驱动能力控制寄存器 **DSC1** 功能：配置控制线的 I/O 驱动能力。
- DSC1 地址 = 0x560000C8，复位值 = 0x00000000，可读可写。

DSC1 寄存器格式 2

DSC_CS6	[13:12]	nGCS6 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_CS5	[11:10]	nGCS5 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_CS4	[9:8]	nGCS4 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_CS3	[7:6]	nGCS3 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_CS2	[5:4]	nGCS2 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_CS1	[3:2]	nGCS1 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00
DSC_CS0	[1:0]	nGCS0 驱动能力。 00=12mA ; 10=10mA ; 01=6mA ; 11=4mA 。	00

5.2.4 I/O 应用实例

- 例 5-2：LED 灯的控制，如图所示，通过编程控制 GPF3、GPF4、GPF5、GPF6 口 4 个引脚的输出电平（高 / 低电平），实现对 LED1、LED2、LED3、LED4 亮灭的控制。
- LED 灯通过电源、限流电阻与 ARM 的 GPIO 的 GPF3~ GPF6 口相连。
- 要求：GPIO 口线 = 低电平时，LED 灯亮。
- GPIO 口线 = 高电平时，LED 灯灭。



DSC0 寄存器格式

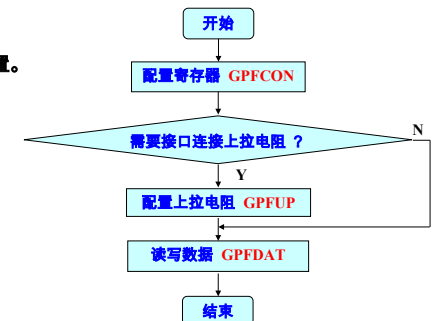
DSC	位	描述	初始值
nEN_DSC	[31]	使能端。0 = 使能 ; 1 = 不使能。	0
Reserved	[30:10]	保留。	0
DSC_ADR	[9:8]	地址总线驱动能力。 00=12mA ; 10=10mA ; 01=8mA ; 11=6mA 。	00
DSC_DATA3	[7:6]	DATA[31:24] 线驱动能力。 00=12mA ; 10=10mA ; 01=8mA ; 11=6mA 。	00
DSC_DATA2	[5:4]	DATA[23:16] 线驱动能力。 00=12mA ; 10=10mA ; 01=8mA ; 11=6mA 。	00
DSC_DATA1	[3:2]	DATA[15:8] 线驱动能力。 00=12mA ; 10=10mA ; 01=8mA ; 11=6mA 。	00
DSC_DATA0	[1:0]	DATA[7:0] 线驱动能力。 00=12mA ; 10=10mA ; 01=8mA ; 11=6mA 。	00

9. 内存休眠控制寄存器 MSLCON

- 内存休眠：把内存的数据直接写在硬盘上，然后内存断电。睡眠是只给内存供电，其他部件都断电。
- 内存休眠控制寄存器 **MSLCON**：设置在休眠状态下存储器接口信号的状态。
- MSLCON 地址 = 0x560000CC，复位值 = 0x0，可读可写。

针对 GPF 的编程

- GPF 编程步骤：
 - GPF 引脚功能配置。
 - GPF 引脚上拉电阻配置。
 - GPF 引脚读写数据。



GPF 引脚功能配置

- 配置 GPFCON (地址 0x56000050) : 例如 GPF3-GPF0 输入, GPF7-GPF4 输出。
- 01 01 01 01 → GPF7-GPF4, 00 00 00 00 → GPF3-GPF0
- GPFCON 寄存器值 →
GPF7-GPF0 01 01 01 01 00 00 00 00 = 0x5500
- 写入寄存器:

C 语言:

```
rGPFCON = 0x5500;
```

汇编语言:

```
GPFCONC EQU 0x5500
```

```
GPFCON EQU 0x56000050
```

```
LDR r0, =GPFCONC
```

```
LDR r1, =GPFCON
```

```
STR r0, [r1]
```

GPFCON	位	描述
GPF7	[15:14]	00=输入; 01=输出; 10=EINT[7]; 11=保留。
GPF6	[13:12]	00=输入; 01=输出; 10=EINT[6]; 11=保留。
GPF5	[11:10]	00=输入; 01=输出; 10=EINT[5]; 11=保留。
GPF4	[9:8]	00=输入; 01=输出; 10=EINT[4]; 11=保留。
GPF3	[7:6]	00=输入; 01=输出; 10=EINT[3]; 11=保留。
GPF2	[5:4]	00=输入; 01=输出; 10=EINT[2]; 11=保留。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY

例 5-2 : GPF 引脚上拉电阻配置

- 配置 GPFUP (地址 0x56000058) : 本例 GPF6-GPF3 配置上拉电阻, 其他位 GPF7、GPF2-GPF0 配置不变。
- 0000 → GPF6-GPF3
- GPFUP 寄存器值: GPF7-GPF0 *0000*** (*表示不变)
- 写入寄存器:
rGPFUP&=0x87; // 按位“与”后赋值, 0x87 = 1000 0111

GPFUP	位	描述
GPF 7:0	[7:0]	1=对应的 GPIO 引脚上拉电阻不使能; 0=对应的 GPIO 引脚上拉电阻使能。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

85

LED 灯控制程序 1

```
#define rGPFCON (*(volatile unsigned *)0x56000050) // 定义 GPFCON 地址
#define rGPFDAT (*(volatile unsigned *)0x56000054) // 定义 GPFDAT 地址
#define rGPFUP (*(volatile unsigned *)0x56000058) // 定义 GPFUP 地址
void Delay(unsigned int); // 无返回结果, LED 点亮的延时程序
int Ledlamp() // 流水灯控制程序
{
    unsigned char ledtab[]={ 0xf7, 0xef, 0xdf, 0xbf }; // 流水灯点亮数据
    int i;
    rGPFUP&=0x87; // 配置 GPF6-GPF3 使用上拉电阻
    rGPFCON&=0xC03f;
    rGPFCON|=0x1540; // 配置 GPF6-GPF3 引脚为输出
    while (1)
    {
        for (i=0; i<4; i++) // 循环点亮 4 个发光二极管
        {
            rGPFDAT=ledtab[i]; // 点亮发光二极管 LEDi
            Delay(70); // 延时
        }
    }
    return(0);
}
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

88

例 5-2 : GPF 引脚功能配置

- 配置 GPFCON (地址 0x56000050) : 本例 GPF6-GPF3 输出, GPF7、GPF2-GPF0 不变。 01 01 01 01 → GPF6-GPF3
- GPFCON 寄存器值: GPF7-GPF0 = ** 01 01 01 01 ** *** (*表示不变)
- 写入寄存器:
- rGPFCON&=0xC03F; // 按位“与”后赋值, 0xC03F=11 00 00 00 00 11 11 11
- rGPFCON|=0x1540; // 按位“或”后赋值, 0x1540=00 01 01 01 00 00 00 00

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY

GPF 引脚读写数据

- 读写数据 GPFDAT (地址 0x56000054)

C 语言

读数据:

```
Var=rGPFDAT
```

写数据:

```
rGPFDAT=Var
```

汇编语言

读数据:

```
GPFDAT EQU 0x56000054
LDR R0, =GPFDAT
LDR R1, [R0]
```

写数据:

```
GPFDAT EQU 0x56000054
LDR R0, =GPFDAT
MOV R1, #0x80
STR R1, [R0]
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

86

LED 灯控制程序 2

```
void Delay (unsigned int x) // 延时程序
{
    unsigned int i,j,k;
    for(i=0;i<=x;i++)
        for(j=0;j<=0xff;j++)
            for(k=0;k<=0xff;k++);
}

AREA [DATA], CODE, READONLY ; 主程序
ENTRY
LER R13, =0x1000 ; 设置堆栈地址指针, 子程序调用时保护现场
BL Ledlamp ; 调用 C 语言的流水灯函数 Ledlamp
END
IMPORT Ledlamp ; 声明标号 Ledlamp 是在外部定义的, 即在其他的源文件中定义的
```

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

89

GPF 引脚上拉电阻配置

- 配置 GPFUP (地址 0x56000058) : 例如 GPF7-GPF0 配置上拉电阻。
- 0000 0000 → GPF7-GPF0
- GPFUP 寄存器值 → GPF7-GPF0 00000000=0x00
- 写入寄存器:

C 语言:

```
rGPFUP = 0x00;
```

汇编语言:

```
GPFUP EQU 0x56000058
```

```
LDR R0, =GPFUP
```

```
MOV R1, #0x00
```

```
STR R1, [R0]
```

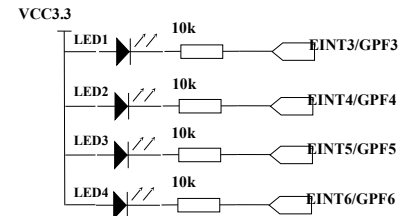
GPFUP	位	描述
GPF 7:0	[7:0]	1=对应的 GPIO 引脚上拉电阻不使能; 0=对应的 GPIO 引脚上拉电阻使能。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

84

例 5-2 : LED 灯的控制

- GPIO 口线 = 低电平, LED 灯亮; GPIO 口线 = 高电平, LED 灯灭。
- LED1 亮: GPF7-GPF0 = 1111 0111=0xF7, GPFDAT=0xF7
- LED2 亮: GPF7-GPF0 = 1110 1111=0xEF, GPFDAT=0xEF
- LED3 亮: GPF7-GPF0 = 1101 1111=0xDF, GPFDAT=0xDF
- LED4 亮: GPF7-GPF0 = 1011 1111=0xBF, GPFDAT=0xBF



Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

87

5.3 中断系统

- S3C2440 内部有一个中断控制器, 用于管理 60 个中断源的中断过程。
- 中断控制器作用:
 - (1) 响应 FIQ (快速中断请求) 或 IRQ (普通中断请求) 的中断请求。
 - (2) 请求内核对中断进行处理。
 - (3) 中断优先级排队。
- 中断模式: FIQ、IRQ
- IRQ 和 FIQ 的区别:
 - (1) 对于 FIQ, 必须尽快处理事件并离开这个模式。
 - (2) IRQ 可以被 FIQ 中断, 但 IRQ 不能中断 FIQ。
 - (3) 为使 FIQ 更快, FIQ 模式具有更多的私有寄存器。
- CPSR 状态寄存器影响中断的处理:
 - F=1, 处理器不响应 FIQ; F=0, 处理器响应 FIQ。
 - I=1, 处理器不响应 IRQ; I=0, 处理器响应 IRQ。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn

90

中断源

- S3C2440 有 **60 个中断源**。每一个中断源对应寄存器中的一位，显然要支持 60 个中断源至少需要 2 个 32 位寄存器，S3C2440 用多个寄存器保存中断源信号。
- 中断源类型：内部中断源、外部中断源。
 - (1) **内部中断源**：来自 S3C2440 片内的中断源，**36 个**，很多片内接口部件可以产生中断，如 DMA、UART、ADC、LCD、定时器、USB、IIC 等。
 - (2) **外部中断源**：来自 S3C2440 片外的中断源，**24 个**，片外设备可以通过引脚 EINT23-EINT0 产生中断，如键盘、蓝牙、传感器、WiFi 等设备模块。
- EINT23-EINT0：24 个外部中断占用 GPF0-GPF7（EINT0-EINT7），GPG0-GPG15（EINT8-EINT23）。使用时，必须配置为中断，并且不要上拉电阻。
- 外部中断涉及的寄存器：（已经在 GPIO 中介绍了）
 - (1) **EXTINT0-EXTINT2**：分别设置 EINT0-EINT7、EINT8-EINT15、EINT16-EINT23 的触发方式（高电平触发、低电平触发、下降沿触发、上升沿触发）。
 - (2) **EINTFLT0-EINTFLT3**：控制滤波时钟和滤波宽度。
 - (3) **EINTPEND**：外部中断未决寄存器，保存 EINT4-EINT23 的中断请求状态，1=有中断请求。
 - (4) **EINTMSK**：外部中断屏蔽寄存器，1=屏蔽相应中断。

内部一级中断源

- 19 个内部一级中断源
 - 1 个 RTC 中断源 INT_RTC
 - 2 个 SPI 中断源 INT_SPIO,INT_SPII
 - 1 个 IIC 中断源 INT_IIC
 - 2 个 USB 中断源 INT_USBH,INT_USBD
 - 1 个 Nand Flash 中断源 INT_NFCON
 - 1 个 SDI 中断源 INT_SDI
 - 4 个 DMA 中断源 INT_DMA3,INT_DMA2,INT_DMA1,INT_DMA0
 - 5 个定时器中断源 INT_TIMER4,INT_TIMER3,INT_TIMER2,INT_TIMER1,INT_TIMER0
 - 1 个时钟节拍中断源 INT_TICK
 - 1 个电池故障中断源 nBATT_FLT
- 7 个内部复合中断源
 - *1 个 ADC 中断源 INT_ADC（内含 2 个二级中断源）
 - *3 个 UART 中断源 INT_UART0,INT_UART1,INT_UART2（9 个二级中断源）
 - *1 个 LCD 中断源 INT_LCD（内含 2 个二级中断源）
 - *1 个看门狗和 AC97 中断源 INT_WDT_AC97（内含 2 个二级中断源）
 - *1 个摄像头中断源 INT_CAM（内含 2 个二级中断源）

中断控制器支持的中断源 1

中断源名称	描述	仲裁组
INT_ADC	ADC 结束中断、触摸屏中断，2 个子中断源	ARB5
INT_RTC	RTC 闹钟中断	ARB5
INT_SPIO	SPIO 中断	ARB5
INT_UART0	串口 0 中断（ERR、RXD、TXD），3 个子中断源	ARB5
INT_IIC	IIC 中断	ARB4
INT_USBH	USB 主机中断	ARB4
INT_USBD	USB 设备中断	ARB4
INT_NFCON	Nand Flash 控制中断	ARB4
INT_UART1	串口 1 中断（ERR、RXD、TXD），3 个子中断源	ARB4
INT_SPIO	SPIO 中断	ARB4
INT_SDI	SDI 中断	ARB3
INT_DMA3	DMA 通道 3 中断	ARB3
INT_DMA2	DMA 通道 2 中断	ARB3
INT_DMA1	DMA 通道 1 中断	ARB3
INT_DMA0	DMA 通道 0 中断	ARB3

SRCPND 中的中断源类型

- S3C2440 中断控制器中有一个 **SRCPND 寄存器（中断源未决寄存器）**，保存中断源申请中断的状态（申请中断、未申请中断），SRCPND 中的中断请求可以通过中断屏蔽、中断模式、中断优先级处理，直接向 ARM 核申请中断。
- SRCPND 中的中断源类型：直接中断源、复合中断源。
- 直接中断源：来源单一的中断源，如 INT_RTC（RTC 闹钟中断）、EINT0（外部中断 0）等。一旦中断请求被响应，可以直接定位中断来源。
- 复合中断源：由多个中断源复合在一起的中断源，实际上是一个由多个中断源的中断请求以“或”逻辑复合在一起的中断请求标志。由于该标志由多个中断请求复合而成，所以一旦中断请求被响应，需要通过其他方法定位中断来源。如 INT_UART0（串口 0 中断）等。
- 一级中断源：SRCPND 中的直接中断源，23 个（内部中断源 19+ 外部中断源 4）。
- 二级中断源：SRCPND 中的复合中断源涉及的中断源，37 个（内部中断源 17+ 外部中断源 20）。

内部二级中断

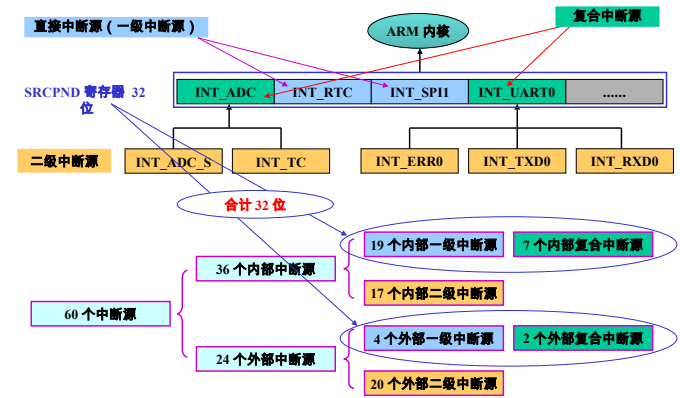
17 个内部二级中断源

3 个 UART0 二级中断源： INT_RXD0 数据接收中断 INT_TXD0 数据传输中断 INT_ERR0 UART 错误中断	2 个 ADC 二级中断源： INT_ADC_S ADC 转换中断 INT_TC 触摸屏中断（按
3 个 UART1 二级中断源： INT_RXD1 数据接收中断 INT_TXD1 数据传输中断 INT_ERR1 UART 错误中断	2 个看门狗和 AC97 二级中断源： INT_WDT 看门狗中断 INT_AC97 AC97 中断
3 个 UART2 二级中断源： INT_RXD2 数据接收中断 INT_TXD2 数据传输中断 INT_ERR2 UART 错误中断	2 个摄像头二级中断源： INT_CAM_C 摄像头 C 端口捕捉中断
	2 个 LCD 二级中断源： INT_FrSyn LCD 帧同步中断 INT_FiCnt LCD 的 FIFO 中断

中断控制器支持的 60 个中断源 2

INT_LCD	LCD 中断（INT_FrSyn、INT_FiCnt），2 个子中断源	ARB3
INT_UART2	串口 2 中断（ERR、RXD、TXD），3 个子中断源	ARB2
INT_TIMER4	定时器 4 中断	ARB2
INT_TIMER3	定时器 3 中断	ARB2
INT_TIMER2	定时器 2 中断	ARB2
INT_TIMER1	定时器 1 中断	ARB2
INT_TIMER0	定时器 0 中断	ARB2
INT_WDT_AC97	看门狗定时器中断，2 个子中断源	ARB1
INT_TICK	RTC 定时中断	ARB1
nBATT_FLT	电池失效中断	ARB1
INT_CAM	摄像头接口，2 个子中断源	ARB1
EINT8_23	外部中断 8_23，16 个子中断源	ARB1
EINT4_7	外部中断 4_7，4 个子中断源	ARB1
EINT3	外部中断 3	ARB0
EINT2	外部中断 2	ARB0
EINT1	外部中断 1	ARB0
EINT0	外部中断 0	ARB0

中断源类型



外部中断

4 个外部一级中断源

- EINT0
- EINT1
- EINT2
- EINT3

2 个外部复合中断源

- EINT4_7
- EINT8_23

20 个外部二级中断源

- EINT4_7：EINT4-EINT7
- EINT8_23：EINT8-EINT23

- EINT0-EINT7 对应 GPF0-GPF7，共 8 个
- EINT8-EINT23 对应 GPG0-GPG15，共 16 个
- 对应的 I/O 口设为中断类型
- 禁止该 I/O 口上拉电阻功能

子中断源（内部二级中断源）

中断源名称	描述	源
INT_AC97	AC97 中断	INT_WDT_AC97
INT_WDT	WDT 中断	INT_WDT_AC97
INT_CAM_P	摄像头接口的 P 口捕获中断	INT_CAM
INT_CAM_C	摄像头接口的 C 口捕获中断	INT_CAM
INT_ADC_S	ADC 中断	INT_ADC
INT_TC	触摸屏中断（开/关）	INT_ADC
INT_ERR2	UART2 错误中断	INT_UART2
INT_TXD2	UART2 发送中断	INT_UART2
INT_RXD2	UART2 接收中断	INT_UART2
INT_ERR1	UART1 错误中断	INT_UART1
INT_TXD1	UART1 发送中断	INT_UART1
INT_RXD1	UART1 接收中断	INT_UART1
INT_ERR0	UART0 错误中断	INT_UART0
INT_TXD0	UART0 发送中断	INT_UART0
INT_RXD0	UART0 接收中断	INT_UART0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 100



```

graph TD
    subgraph Inputs
        A[内部一般中断源] --> B[SUBSRCPND]
        C[内部二级中断源  
(除 LCD 之外)] --> D[LCDSRCPND]
        E[外部一般中断源  
EINT0-EINT3] --> F[EINTPND]
    end

    B --> G[SUBINTMSK]
    D --> H[LCDINTMSK]
    F --> I[EINTMSK]

    G --> J[LCDINTPND]
    H --> J
    I --> K[外部中断未决寄存器]

    J --> L[SRCPND 位[4]=1]
    K --> L

    L --> M[中断屏蔽寄存器  
INTMSK 位[4]=0, 允许]
    M --> N[中断模式寄存器  
INTMOD]
    N --> O[PRIORITY]
    O --> P[INTMOD 位[4]=0, EINT4_7 选 IRQ]
    O --> Q[INTOFFSET 中断偏移寄存器]
    P --> R[FIO]
    Q --> S[INTOFFSET=4, 表明产生中断的是 EINT4_7]
    
```

优先级总规则 (无论是固定优先级, 还是循环优先级): 对于每个仲裁模块, REQ0、REQ5 的优先级**不变**, 并且 REQ0 最高、REQ5 最低。

对于整个仲裁逻辑: 仲裁模块 **ARBITER0** 优先级总是**最高**, **ARBITER5** 优先级总是**最低**, **ARBITER1-ARBITER4** 可能改变。

对于 **ARBITER1**: EINT4 7 优先级总是**最高**, **INT_WDT/AC9** 优先级总是**最低**, **EINT8_23**、**INT_CAM**、**nBATT_FLT**、**INT_TICK** 可能改变。

系统复位后 **EINT0** **最高**, **INT_ADC** **最低**。

Arbiter	Priority List (Highest to Lowest)
ARBITER1	- REQ2INT_CAM - REQ3BATT_FLT - REQ4INT_TICK - REQ5INT_WDT/AC9
ARBITER2	- REQ0INT_TIMER0 - REQ1INT_TIMER1 - REQ2INT_TIMER2 - REQ3INT_TIMER3 - REQ4INT_TIMER4 - REQ5INT_UART2
ARBITER3	- REQ0INT_LCD - REQ1INT_DMA0 - REQ2INT_DMA1 - REQ3INT_DMA2 - REQ4INT_DMA3 - REQ5INT_SDI
ARBITER4	- REQ0INT_SPIO - REQ1INT_UART1 - REQ2INT_NFCON - REQ3INT_USH - REQ4INT_USHIB - REQ5INT_IIC
ARBITER5	- REQ1INT_UART0 - REQ2INT_SPI1 - REQ3INT_RTC - REQ4INT_IIC

(- IRQ)
 GY zhaohw@jlu.edu.cn 101



```

graph TD
    Start([开始中断服务]) --> IntSet[中断中断置位 INTTOFSET]
    IntSet --> L1[一级中断]
    IntSet --> L2[内部二级中断]
    IntSet --> L2LCD[内部二级中断 LCD]
    IntSet --> L2Ext[外部二级中断]

    L1 --> Serv1[中断服务]
    Serv1 --> ClearSRC[清除 SRCPND 位]

    L2 --> CheckSub{SUBSRCPND  
每一位检测完毕}
    CheckSub -- 是 --> Serv2[中断服务]
    Serv2 --> ClearSub[清除 SUBSRCPND 位]
    ClearSub --> ClearSRC

    CheckSub -- 否 --> CheckSubSRC{SUBSRCPND 位为 1,  
SUBINTMSK 对应位为 0}
    CheckSubSRC -- 是 --> Serv2
    CheckSubSRC -- 否 --> ClearSRC

    L2LCD --> CheckLcd{LCDINTPND  
每一位检测完毕}
    CheckLcd -- 是 --> Serv3[中断服务]
    Serv3 --> ClearLcd[清除 LCDINTPND 位]
    ClearLcd --> ClearSRC

    CheckLcd -- 否 --> Serv3
    Serv3 --> ClearSRC

    L2Ext --> CheckExt{EINTPND  
每一位检测完毕}
    CheckExt -- 是 --> Serv4[中断服务]
    Serv4 --> ClearExt[清除 EINTPND 位]
    ClearExt --> ClearSRC

    CheckExt -- 否 --> CheckExtSRC{EINTPND 位为 1, 且  
EINTMSK 对应位为 0}
    CheckExtSRC -- 是 --> Serv4
    CheckExtSRC -- 否 --> ClearSRC

    ClearSRC --> ClearInt[清除 INTPND 位]
    ClearInt --> End([结束])
  
```

• **固定优先级**：当 **ARB_MODE=0** 时，设置为固定优先级。优先级一旦设定将不再改变。

• **仲裁选择** **ARB_SEL** 用于**设定 REQ1、REQ2、REQ3、REQ4 中的最低优先级**。改变 **ARB_SEL** 的值，可以改变 REQ1 到 REQ4 的优先级。

• 对于 REQ1、REQ2、REQ3、REQ4 来说，优先级顺序是一种**循环式顺序**，即

→ REQ1 → REQ2 → REQ3 → REQ4 →

ARB_SEL	固定优先级（高→低）
00	REQ0 → REQ1 → REQ2 → REQ3 → REQ4 → REQ5
01	REQ0 → REQ2 → REQ3 → REQ4 → REQ1 → REQ5
10	REQ0 → REQ3 → REQ4 → REQ1 → REQ2 → REQ5
11	REQ0 → REQ4 → REQ1 → REQ2 → REQ3 → REQ5

↑ 总是最高
↑ 总是最低

一级仲裁

SRCPND 优

- ARBITER0: REQ0/INT0, REQ0/INT1, REQ0/INT2, REQ0/INT3
- ARBITER1: REQ0/INT4, 7, REQ0/INT8, 23, REQ0/INT_CAM, REQ0/INT_BATT_FLT, REQ0/INT_TICK, REQ0/INT_WDWT/AC9
- ARBITER2: REQ0/INT_TIMER0, REQ0/INT_TIMER1, REQ0/INT_TIMER2, REQ0/INT_TIMER3, REQ0/INT_TIMER4, REQ0/INT_UA12
- ARBITER3: REQ0/INT_LCD, REQ0/INT_DMA0, REQ0/INT_DMA1, REQ0/INT_DMA2, REQ0/INT_DMA3, REQ0/INT_SDI
- ARBITER4: REQ0/INT_SP10, REQ0/INT_UART1, REQ0/INT_NFCON, REQ0/INT_USBD, REQ0/INT_USBH, REQ0/INT_JIC
- ARBITER5: REQ0/INT_UART0, REQ0/INT_SPI, REQ0/INT_RHC, REQ0/INT_ADC

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 102



- **SRCPND**（中断源未决寄存器）中的 **32** 个中断请求通过 **7** 个仲裁器（优先级判别逻辑）的选择，最终生成一个优先级最高的中断源，设置在 **INTPND**（中断未决寄存器）中。
- **仲裁器功能**：确定中断源的优先级，**6** 个一级仲裁器、**1** 个二级仲裁器。
- **优先级仲裁过程**：
 - （1）**先一级**：每个一级仲裁器对自己的分组进行优先级判别，选择一个优先级最高的中断，送给二级仲裁器。**6** 个一级仲裁器形成 **6** 个中断源。
 - （2）**后二级**：二级仲裁器在一级仲裁器输出的 **6** 个中断源中选择一个优先级最高的中断源进行触发。
- **仲裁器控制**：每个仲裁器有 **2** 个控制信号，**ARB_MODE**、**ARB_SEL**。
- **ARB_MODE**：**仲裁模式**，**1** 位。**0**= 固定优先级，**1**= 循环优先级。
- **ARB_SEL**：**仲裁选择**，指出**最低优先级**，**2** 位。
01=REQ1， 10=REQ2， 11=REQ3， 00=REQ4。

• **循环优先级**：ARB_MODE=1，设为循环优先级。

• 此时 ARB_SEL 变成**状态指示**，根据当前处理中断情况**自动改变**，反映 REQ1、REQ2、REQ3、REQ4 中的**最低优先级**。例如如果 REQ1 被处理，则 ARB_SEL 位自动的变为 01，把 REQ1 放到最低的优先级。

• REQ1-REQ4 的循环式优先级顺序，仍然是

→ REQ1 → REQ2 → REQ3 → REQ4 →

ARM 响应的中断	ARB_SEL 自动改变	循环优先级（高→低）
REQ0	00	REQ0→REQ1→REQ2→REQ3→REQ4→REQ5
REQ1	01	REQ0→REQ2→REQ3→REQ4→REQ1→REQ5
REQ2	10	REQ0→REQ3→REQ4→REQ1→REQ2→REQ5
REQ3	11	REQ0→REQ4→REQ1→REQ2→REQ3→REQ5

5.3.2 中断控制寄存器

- 中断源未决寄存器（SRCPND）
- 中断模式寄存器（INTMOD）
- 中断屏蔽寄存器（INTMSK）
- 中断优先级寄存器（PRIORITY）
- 中断未决寄存器（INTPND）
- 中断偏移寄存器（INTOFFSET）
- 子中断源未决寄存器（SUBSRCPND）
- 子中断屏蔽寄存器（SUBINTMSK）
- 外部中断未决寄存器（EINTPND）
- 外部中断屏蔽寄存器（EINTMSK）
- LCD 中断源未决寄存器（LCDSRCPND）
- LCD 中断屏蔽寄存器（LCDINTMSK）
- LCD 中断未决寄存器（LCDINTPND）

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 109

SRCPND 寄存器格式 2

INT_LCD	[16]	确定 INT_LCD 中断请求。0= 没有请求；1= 请求。	0
INT_UART2	[15]	确定 INT_UART2 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER4	[14]	确定 INT_TIMER4 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER3	[13]	确定 INT_TIMER3 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER2	[12]	确定 INT_TIMER2 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER1	[11]	确定 INT_TIMER1 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER0	[10]	确定 INT_TIMER0 中断请求。0= 没有请求；1= 请求。	0
INT_WDT_AC97	[9]	确定 INT_WDT_AC97 中断请求。0= 没有请求；1= 请求。	0
INT_TICK	[8]	确定 INT_TICK 中断请求。0= 没有请求；1= 请求。	0
nBATT_FLT	[7]	确定 nBATT_FLT 中断请求。0= 没有请求；1= 请求。	0
INT_CAM	[6]	确定 INT_CAM 中断请求。0= 没有请求；1= 请求。	0
EINT8_23	[5]	确定 EINT8_23 中断请求。0= 没有请求；1= 请求。	0
EINT4_7	[4]	确定 EINT4_7 中断请求。0= 没有请求；1= 请求。	0
EINT3	[3]	确定 EINT3 中断请求。0= 没有请求；1= 请求。	0
EINT2	[2]	确定 EINT2 中断请求。0= 没有请求；1= 请求。	0
EINT1	[1]	确定 EINT1 中断请求。0= 没有请求；1= 请求。	0
EINT0	[0]	确定 EINT0 中断请求。0= 没有请求；1= 请求。	0

INTMOD 寄存器格式 2

INT_LCD	[16]	确定 INT_LCD 中断请求。0=IRQ；1=FIQ。	0
INT_UART2	[15]	确定 INT_UART2 中断请求。0=IRQ；1=FIQ。	0
INT_TIMER4	[14]	确定 INT_TIMER4 中断请求。0=IRQ；1=FIQ。	0
INT_TIMER3	[13]	确定 INT_TIMER3 中断请求。0=IRQ；1=FIQ。	0
INT_TIMER2	[12]	确定 INT_TIMER2 中断请求。0=IRQ；1=FIQ。	0
INT_TIMER1	[11]	确定 INT_TIMER1 中断请求。0=IRQ；1=FIQ。	0
INT_TIMER0	[10]	确定 INT_TIMER0 中断请求。0=IRQ；1=FIQ。	0
INT_WDT_AC97	[9]	确定 INT_WDT_AC97 中断请求。0=IRQ；1=FIQ。	0
INT_TICK	[8]	确定 INT_TICK 中断请求。0=IRQ；1=FIQ。	0
nBATT_FLT	[7]	确定 nBATT_FLT 中断请求。0=IRQ；1=FIQ。	0
INT_CAM	[6]	确定 INT_CAM 中断请求。0=IRQ；1=FIQ。	0
EINT8_23	[5]	确定 EINT8_23 中断请求。0=IRQ；1=FIQ。	0
EINT4_7	[4]	确定 EINT4_7 中断请求。0=IRQ；1=FIQ。	0
EINT3	[3]	确定 EINT3 中断请求。0=IRQ；1=FIQ。	0
EINT2	[2]	确定 EINT2 中断请求。0=IRQ；1=FIQ。	0
EINT1	[1]	确定 EINT1 中断请求。0=IRQ；1=FIQ。	0
EINT0	[0]	确定 EINT0 中断请求。0=IRQ；1=FIQ。	0

1. 中断源未决寄存器 SRCPND

- SRCPND：中断源未决寄存器，32 位，每位与一个中断源相关。
- 功能：保存中断源申请中断请求的状态，1= 有中断请求，0= 没有中断请求。
- 说明：SRCPND 的位由中断源自动设置，与 INTMSK 是否屏蔽无关。
- SRCPND 位的清除：就是把相应置 1 的位清零，需要用指令清除。响应中断之后，SRCPND 相应置 1 的位必须清除，否则会再次响应中断。
- SRCPND 位清除方法：向指定位写 1。
- SRCPND 地址 =0x4A000000，复位值 =0x0000 0000。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 110

2. 中断模式寄存器 INTMOD

- INTMOD：中断模式寄存器，32 位，对应 SRCPND 的每一位。
- 功能：将中断请求设置为 IRQ 或者 FIQ，1=FIQ，0=IRQ。
- 说明：只有一个中断源能工作在 FIQ 模式下，也就是说，INTMOD 只有一个位可以被置 1。
- INTMOD 地址 =0x4A000004，复位值 =0x00000000。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 113

3. 中断屏蔽寄存器 INTMSK

- INTMSK：中断屏蔽寄存器，32 位，每位与一个中断源相关。
- 功能：设置中断屏蔽状态，1= 屏蔽掉相应中断请求，0= 允许。
- INTMSK 地址 =0x4A000008，复位值 =0xFFFF FFFF（复位状态为屏蔽）。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 116

SRCPND 寄存器格式 1

SRCPND	位	描述	初始值
INT_ADC	[31]	确定 INT_ADC 中断请求。0= 没有请求；1= 请求。	0
INT_RTC	[30]	确定 INT_RTC 中断请求。0= 没有请求；1= 请求。	0
INT_SPI1	[29]	确定 INT_SPI1 中断请求。0= 没有请求；1= 请求。	0
INT_UART0	[28]	确定 INT_UART0 中断请求。0= 没有请求；1= 请求。	0
INT_IIC	[27]	确定 INT_IIC 中断请求。0= 没有请求；1= 请求。	0
INT_USBH	[26]	确定 INT_USBH 中断请求。0= 没有请求；1= 请求。	0
INT_USBD	[25]	确定 INT_USBD 中断请求。0= 没有请求；1= 请求。	0
INT_NFCON	[24]	确定 INT_NFCON 中断请求。0= 没有请求；1= 请求。	0
INT_UART1	[23]	确定 INT_UART1 中断请求。0= 没有请求；1= 请求。	0
INT_SPI0	[22]	确定 INT_SPI0 中断请求。0= 没有请求；1= 请求。	0
INT_SDI	[21]	确定 INT_SDI 中断请求。0= 没有请求；1= 请求。	0
INT_DMA3	[20]	确定 INT_DMA3 中断请求。0= 没有请求；1= 请求。	0
INT_DMA2	[19]	确定 INT_DMA2 中断请求。0= 没有请求；1= 请求。	0
INT_DMA1	[18]	确定 INT_DMA1 中断请求。0= 没有请求；1= 请求。	0
INT_DMA0	[17]	确定 INT_DMA0 中断请求。0= 没有请求；1= 请求。	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 111

INTMOD 寄存器格式 1

INTMOD	位	描述	初始值
INT_ADC	[31]	确定 INT_ADC 中断请求。0=IRQ；1=FIQ。	0
INT_RTC	[30]	确定 INT_RTC 中断请求。0=IRQ；1=FIQ。	0
INT_SPI1	[29]	确定 INT_SPI1 中断请求。0=IRQ；1=FIQ。	0
INT_UART0	[28]	确定 INT_UART0 中断请求。0=IRQ；1=FIQ。	0
INT_IIC	[27]	确定 INT_IIC 中断请求。0=IRQ；1=FIQ。	0
INT_USBH	[26]	确定 INT_USBH 中断请求。0=IRQ；1=FIQ。	0
INT_USBD	[25]	确定 INT_USBD 中断请求。0=IRQ；1=FIQ。	0
INT_NFCON	[24]	确定 INT_NFCON 中断请求。0=IRQ；1=FIQ。	0
INT_UART1	[23]	确定 INT_UART1 中断请求。0=IRQ；1=FIQ。	0
INT_SPI0	[22]	确定 INT_SPI0 中断请求。0=IRQ；1=FIQ。	0
INT_SDI	[21]	确定 INT_SDI 中断请求。0=IRQ；1=FIQ。	0
INT_DMA3	[20]	确定 INT_DMA3 中断请求。0=IRQ；1=FIQ。	0
INT_DMA2	[19]	确定 INT_DMA2 中断请求。0=IRQ；1=FIQ。	0
INT_DMA1	[18]	确定 INT_DMA1 中断请求。0=IRQ；1=FIQ。	0
INT_DMA0	[17]	确定 INT_DMA0 中断请求。0=IRQ；1=FIQ。	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 114

INTMSK 寄存器格式 1

INTMSK	位	描述	初始值
INT_ADC	[31]	确定 INT_ADC 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_RTC	[30]	确定 INT_RTC 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_SPI1	[29]	确定 INT_SPI1 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_UART0	[28]	确定 INT_UART0 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_IIC	[27]	确定 INT_IIC 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_USBH	[26]	确定 INT_USBH 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_USBD	[25]	确定 INT_USBD 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_NFCON	[24]	确定 INT_NFCON 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_UART1	[23]	确定 INT_UART1 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_SPI0	[22]	确定 INT_SPI0 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_SDI	[21]	确定 INT_SDI 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_DMA3	[20]	确定 INT_DMA3 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_DMA2	[19]	确定 INT_DMA2 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_DMA1	[18]	确定 INT_DMA1 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_DMA0	[17]	确定 INT_DMA0 中断请求。0= 允许中断；1= 屏蔽中断。	1

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 117

INTMSK 寄存器格式 2

INT_LCD	[16]	确定 INT_LCD 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_UART2	[15]	确定 INT_UART2 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_TIMER4	[14]	确定 INT_TIMER4 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_TIMER3	[13]	确定 INT_TIMER3 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_TIMER2	[12]	确定 INT_TIMER2 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_TIMER1	[11]	确定 INT_TIMER1 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_TIMER0	[10]	确定 INT_TIMER0 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_WDT_AC97	[9]	确定 INT_WDT_AC97 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_TICK	[8]	确定 INT_TICK 中断请求。0= 允许中断；1= 屏蔽中断。	1
nBATT_FLT	[7]	确定 nBATT_FLT 中断请求。0= 允许中断；1= 屏蔽中断。	1
INT_CAM	[6]	确定 INT_CAM 中断请求。0= 允许中断；1= 屏蔽中断。	1
EINT8_23	[5]	确定 EINT8_23 中断请求。0= 允许中断；1= 屏蔽中断。	1
EINT4_7	[4]	确定 EINT4_7 中断请求。0= 允许中断；1= 屏蔽中断。	1
EINT3	[3]	确定 EINT3 中断请求。0= 允许中断；1= 屏蔽中断。	1
EINT2	[2]	确定 EINT2 中断请求。0= 允许中断；1= 屏蔽中断。	1
EINT1	[1]	确定 EINT1 中断请求。0= 允许中断；1= 屏蔽中断。	1
EINT0	[0]	确定 EINT0 中断请求。0= 允许中断；1= 屏蔽中断。	1

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 118

PRIORITY 寄存器格式 2

ARB_SEL0	[8:7]	确定仲裁判决器 Arbiter 0 的优先级顺序。 00 = REQ 1-2-3-4；01 = REQ 2-3-4-1； 10 = REQ 3-4-1-2；11 = REQ 4-1-2-3。	00
ARB_MODE6	[6]	确定仲裁判决器 Arbiter 6 的循环优先级顺序。 0 = 优先级不循环；1 = 优先级循环。	1
ARB_MODE5	[5]	确定仲裁判决器 Arbiter 5 的循环优先级顺序。 0 = 优先级不循环；1 = 优先级循环。	1
ARB_MODE4	[4]	确定仲裁判决器 Arbiter 4 的循环优先级顺序。 0 = 优先级不循环；1 = 优先级循环。	1
ARB_MODE3	[3]	确定仲裁判决器 Arbiter 3 的循环优先级顺序。 0 = 优先级不循环；1 = 优先级循环。	1
ARB_MODE2	[2]	确定仲裁判决器 Arbiter 2 的循环优先级顺序。 0 = 优先级不循环；1 = 优先级循环。	1
ARB_MODE1	[1]	确定仲裁判决器 Arbiter 1 的循环优先级顺序。 0 = 优先级不循环；1 = 优先级循环。	1
ARB_MODE0	[0]	确定仲裁判决器 Arbiter 0 的循环优先级顺序。 0 = 优先级不循环；1 = 优先级循环。	1

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 121

INTPND 寄存器格式 2

INT_LCD	[16]	确定 INT_LCD 中断请求。0= 没有请求；1= 请求。	0
INT_UART2	[15]	确定 INT_UART2 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER4	[14]	确定 INT_TIMER4 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER3	[13]	确定 INT_TIMER3 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER2	[12]	确定 INT_TIMER2 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER1	[11]	确定 INT_TIMER1 中断请求。0= 没有请求；1= 请求。	0
INT_TIMER0	[10]	确定 INT_TIMER0 中断请求。0= 没有请求；1= 请求。	0
INT_WDT_AC97	[9]	确定 INT_WDT_AC97 中断请求。0= 没有请求；1= 请求。	0
INT_TICK	[8]	确定 INT_TICK 中断请求。0= 没有请求；1= 请求。	0
nBATT_FLT	[7]	确定 nBATT_FLT 中断请求。0= 没有请求；1= 请求。	0
INT_CAM	[6]	确定 INT_CAM 中断请求。0= 没有请求；1= 请求。	0
EINT8_23	[5]	确定 EINT8_23 中断请求。0= 没有请求；1= 请求。	0
EINT4_7	[4]	确定 EINT4_7 中断请求。0= 没有请求；1= 请求。	0
EINT3	[3]	确定 EINT3 中断请求。0= 没有请求；1= 请求。	0
EINT2	[2]	确定 EINT2 中断请求。0= 没有请求；1= 请求。	0
EINT1	[1]	确定 EINT1 中断请求。0= 没有请求；1= 请求。	0
EINT0	[0]	确定 EINT0 中断请求。0= 没有请求；1= 请求。	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 124

4. 中断优先级寄存器 PRIORITY

- PRIORITY**：中断优先级寄存器，32 位。
- 功能**：设置 **IRQ** 中断优先级。
- 中断优先级逻辑共有 7 个仲裁器，6 个一级仲裁器 ARBITER0-ARBITER5，1 个二级仲裁器 ARBITER6。
- 每个仲裁器在 **PRIORITY** 寄存器中有 3 位对应，分别代表 **ARB_MODE** 的 1 位和 **ARB_SEL** 的 2 位。
- ARB_MODE**：仲裁模式，1 位。0= 固定优先级，1= 循环优先级。
- ARB_SEL**：仲裁选择，指出最低优先级，2 位。
01=REQ1，10=REQ2，11=REQ3，00=REQ4。
- SRCPND** 地址 =0x4A00000C，复位值 =0x0000 007F。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 119

5. 中断未决寄存器 INTPND

- INTPND**：中断未决寄存器，32 位，每一位对应一个中断源。
- 功能**：保存通过屏蔽和优先级仲裁的正在申请 **IRQ** 中断的请求状态，1= 有中断请求，0= 没有中断请求。
- 说明**：**INTPND** 寄存器位于优先级仲裁逻辑之后，因此只有 1 位可以被置 1，且中断请求生成对 ARM 核的 **IRQ**。
- 在 **IRQ** 中断服务程序中，可以读取该寄存器决定哪个中断源被处理。该寄存器中的某一位被置 1，表明相应的中断请求有最高优先级、未屏蔽且在等待中断服务。
- INTPND** 位的清除：就是把相应置 1 的位清零，需要用指令清除。响应中断之后，**INTPND** 相应置 1 的位必须清除，否则会再次响应中断。
- INTPND** 位清除方法：向指定位写 1。
- INTPND** 地址 =0x4A000010，复位值 =0x0000 0000。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 122

6. 中断偏移寄存器 INTOFFSET

- INTOFFSET**：中断偏移寄存器。
- 功能**：保存中断未决寄存器 **INTPND** 中置 1 位的位编号值，值范围是 0-31，比如中断 **INT_ADC** 在 **INTPND** 中处在位 [31]，如果 **INT_ADC** 产生中断请求，并在 **INTPND** 中置 1，则 **INTOFFSET** 中保存的值就是 31，是 **INT_ADC** 在 **INTPND** 中位 [31] 的编号值。
- 说明**：通过读取 **INTOFFSET** 的值，就知道 ARM 响应的是哪个中断源，进而控制程序跳转到相应的中断服务程序。
- INTOFFSET** 寄存器可以通过清除 **SRCPND** 和 **INTPND** 寄存器的相应位自动清除。
- INTOFFSET** 地址 =0x4A000014，复位值 =0x0000 0000。只允许读取操作。

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 125

PRIORITY 寄存器格式 1

PRIORITY	位	描述	初始值
ARB_SEL6	[20:19]	确定仲裁判决器 Arbiter 6 的优先级顺序。 00 = REQ 0-1-2-3-4-5；01 = REQ 0-2-3-4-1-5； 10 = REQ 0-3-4-1-2-5；11 = REQ 0-4-1-2-3-5。	00
ARB_SEL5	[18:17]	确定仲裁判决器 Arbiter 5 的优先级顺序。 00 = REQ 1-2-3-4；01 = REQ 2-3-4-1； 10 = REQ 3-4-1-2；11 = REQ 4-1-2-3。	00
ARB_SEL4	[16:15]	确定仲裁判决器 Arbiter 4 的优先级顺序。 00 = REQ 0-1-2-3-4-5；01 = REQ 0-2-3-4-1-5； 10 = REQ 0-3-4-1-2-5；11 = REQ 0-4-1-2-3-5。	00
ARB_SEL3	[14:13]	确定仲裁判决器 Arbiter 3 的优先级顺序。 00 = REQ 0-1-2-3-4-5；01 = REQ 0-2-3-4-1-5； 10 = REQ 0-3-4-1-2-5；11 = REQ 0-4-1-2-3-5。	00
ARB_SEL2	[12:11]	确定仲裁判决器 Arbiter 2 的优先级顺序。 00 = REQ 0-1-2-3-4-5；01 = REQ 0-2-3-4-1-5； 10 = REQ 0-3-4-1-2-5；11 = REQ 0-4-1-2-3-5。	00
ARB_SEL1	[10:9]	确定仲裁判决器 Arbiter 1 的优先级顺序。 00 = REQ 0-1-2-3-4-5；01 = REQ 0-2-3-4-1-5； 10 = REQ 0-3-4-1-2-5；11 = REQ 0-4-1-2-3-5。	00

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 120

INTPND 寄存器格式 1

INTPND	位	描述	初始值
INT_ADC	[31]	确定 INT_ADC 中断请求。0= 没有请求；1= 请求。	0
INT_RTC	[30]	确定 INT_RTC 中断请求。0= 没有请求；1= 请求。	0
INT_SPI1	[29]	确定 INT_SPI1 中断请求。0= 没有请求；1= 请求。	0
INT_UART0	[28]	确定 INT_UART0 中断请求。0= 没有请求；1= 请求。	0
INT_IIC	[27]	确定 INT_IIC 中断请求。0= 没有请求；1= 请求。	0
INT_USBH	[26]	确定 INT_USBH 中断请求。0= 没有请求；1= 请求。	0
INT_USBD	[25]	确定 INT_USBD 中断请求。0= 没有请求；1= 请求。	0
INT_NFCON	[24]	确定 INT_NFCON 中断请求。0= 没有请求；1= 请求。	0
INT_UART1	[23]	确定 INT_UART1 中断请求。0= 没有请求；1= 请求。	0
INT_SPI0	[22]	确定 INT_SPI0 中断请求。0= 没有请求；1= 请求。	0
INT_SDI	[21]	确定 INT_SDI 中断请求。0= 没有请求；1= 请求。	0
INT_DMA3	[20]	确定 INT_DMA3 中断请求。0= 没有请求；1= 请求。	0
INT_DMA2	[19]	确定 INT_DMA2 中断请求。0= 没有请求；1= 请求。	0
INT_DMA1	[18]	确定 INT_DMA1 中断请求。0= 没有请求；1= 请求。	0
INT_DMA0	[17]	确定 INT_DMA0 中断请求。0= 没有请求；1= 请求。	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 123

INTOFFSET 寄存器格式

INTOFFSET	偏移量	INTOFFSET	偏移量	INTOFFSET	偏移量	INTOFFSET	偏移量
INT_ADC	31	INT_UART1	23	INT_UART2	15	nBATT_FLT	7
INT_RTC	30	INT_SPI0	22	INT_TIMER4	14	INT_CAM	6
INT_SPI1	29	INT_SDI	21	INT_TIMER3	13	EINT8_23	5
INT_UART0	28	INT_DMA3	20	INT_TIMER2	12	EINT4_7	4
INT_IIC	27	INT_DMA2	19	INT_TIMER1	11	EINT3	3
INT_USBH	26	INT_DMA1	18	INT_TIMER0	10	EINT2	2
INT_USBD	25	INT_DMA0	17	INT_WDT_AC97	9	EINT1	1
INT_NFCON	24	INT_LCD	16	INT_TICK	8	EINT0	0

Ji Lin University China COMPUTER SCIENCE AND TECHNOLOGY zhaohw@jlu.edu.cn 126

7. 子中断源未决寄存器 SUBSRCPND

- SUBSRCPND**：子中断源未决寄存器，32 位寄存器，15 位有效，寄存器中的每一位对应一个二级中断源（子中断源）。
- 功能**：保存二级中断源（子中断源）申请中断请求的状态，1= 有中断请求，0= 没有中断请求。
- 说明**：SUBSRCPND 的位由子中断源自动设置，与 SUBINTMSK 是否屏蔽无关。
- SUBSRCPND 位的清除**：就是把相应置 1 的位清零，需要用指令清除。响应中断之后，SUBSRCPND 相应置 1 的位必须清除，否则会再次响应中断。
- SUBSRCPND 位清除方法**：向指定位写 1。
- SUBSRCPND 地址** =0x4A000018，复位值 =0x0000 0000。

SUBINTMSK 寄存器格式 1

SUBINTMSK	位	描述	初始值
Reserved	[31:15]	保留	0
INT_AC97	[14]	确定 INT_AC97 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_WDT	[13]	确定 INT_WDT 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_CAM_P	[12]	确定 INT_CAM_P 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_CAM_C	[11]	确定 INT_CAM_C 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_ADC_S	[10]	确定 INT_ADC_S 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_TC	[9]	确定 INT_TC 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_ERR2	[8]	确定 INT_ERR2 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_TXD2	[7]	确定 INT_TXD2 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1

IRQ 中断处理程序转移方法

- 配置中断向量**
 - #define pISR_RESET (* (unsigned *) (_ISR_STARTADDRESS+0x0))
 - #define pISR_UNDEF (* (unsigned *) (_ISR_STARTADDRESS+0x4))
 - #define pISR_SWI (* (unsigned *) (_ISR_STARTADDRESS+0x8))
 - #define pISR_PABORT (* (unsigned *) (_ISR_STARTADDRESS+0xc))
 - #define pISR_DABORT (* (unsigned *) (_ISR_STARTADDRESS+0x10))
 - #define pISR_RESERVED (* (unsigned *) (_ISR_STARTADDRESS+0x14))
 - #define pISR_IRQ (* (unsigned *) (_ISR_STARTADDRESS+0x18))
 - #define pISR_FIQ (* (unsigned *) (_ISR_STARTADDRESS+0x1c))
 - #define pISR_EINT0 (* (unsigned *) (_ISR_STARTADDRESS+0x20))
 - #define pISR_EINT1 (* (unsigned *) (_ISR_STARTADDRESS+0x24))
 - #define pISR_EINT2 (* (unsigned *) (_ISR_STARTADDRESS+0x28))
 -
 -
 - #define pISR_RTC (* (unsigned *) (_ISR_STARTADDRESS+0x98))
 - #define pISR_ADC (* (unsigned *) (_ISR_STARTADDRESS+0x9c))
- 每个中断向量间隔为 4 个字节
- pISR_EINT0 的位置相当于中断向量表的起始地址，HandleEINT0 + INTOFFSET×4=INTPND 中断的中断向量地址
- 相当于一级中断源中断向量表

SUBSRCPND 寄存器格式

SUBSRCPND	位	描述	初始值
Reserved	[31:15]	保留	0
INT_AC97	[14]	0= 没有请求；1= 请求。	0
INT_WDT	[13]	0= 没有请求；1= 请求。	0
INT_CAM_P	[12]	0= 没有请求；1= 请求。	0
INT_CAM_C	[11]	0= 没有请求；1= 请求。	0
INT_ADC_S	[10]	0= 没有请求；1= 请求。	0
INT_TC	[9]	0= 没有请求；1= 请求。	0
INT_ERR2	[8]	0= 没有请求；1= 请求。	0
INT_TXD2	[7]	0= 没有请求；1= 请求。	0
INT_RXD2	[6]	0= 没有请求；1= 请求。	0
INT_ERR1	[5]	0= 没有请求；1= 请求。	0
INT_TXD1	[4]	0= 没有请求；1= 请求。	0
INT_RXD1	[3]	0= 没有请求；1= 请求。	0
INT_ERR0	[2]	0= 没有请求；1= 请求。	0
INT_TXD0	[1]	0= 没有请求；1= 请求。	0
INT_RXD0	[0]	0= 没有请求；1= 请求。	0

SUBINTMSK 寄存器格式 2

INT_RXD2	[6]	确定 INT_RXD2 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_ERR1	[5]	确定 INT_ERR1 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_TXD1	[4]	确定 INT_TXD1 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_RXD1	[3]	确定 INT_RXD1 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_ERR0	[2]	确定 INT_ERR0 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_TXD0	[1]	确定 INT_TXD0 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1
INT_RXD0	[0]	确定 INT_RXD0 中断屏蔽位。0= 允许中断；1= 屏蔽中断。	1

转移到一级中断源的中断服务程序

IRQ_Handler:

```
sub sp, sp, #4 ; 在堆栈中预留一个字单元，保存转移目标的入口地址
stmfd sp!, {r8-r9}; r8、r9 进栈，保护 r8、r9
ldr r9, =INTOFFSET ; 将 INTOFFSET 寄存器地址送 r9
ldr r9, [r9] ; 将 INTOFFSET 中的中断偏移值送 r9，该值代表被响应的中断
ldr r8, =HandleEINT0 ; 将一级中断源的中断向量表首地址送 r8
add r8, r8, r9, lsl #2 ; 计算被响应中断的中断向量地址，r8=r8+r9×4
ldr r8, [r8] ; 读取被响应中断的中断向量（即中断服务程序入口地址）
str r8, [sp, #8] ; 将获取的中断向量送入堆栈预留的字单元
ldmfd sp!, {r8-r9, pc} ; 恢复 r8、r9，中断向量弹出到 PC，ARM 按 PC 取指令，转入被响应中断的中断服务程序
```

8. 子中断屏蔽寄存器 SUBINTMSK

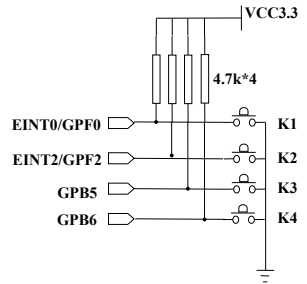
- SUBINTMSK**：子中断屏蔽寄存器，32 位寄存器，15 位有效，每位与一个子中断源相关。
- 功能**：设置子中断源中断屏蔽状态，1= 屏蔽掉相应子中断源中断请求，0= 允许。
- SUBINTMSK 地址** =0x4A00001C，复位值 =0xFFFF（复位状态为屏蔽）。

中断控制寄存器：注意事项

- SRCPND 中断源未决寄存器**：位 =1，表示相应的中断源产生了中断请求。位 =0，表示相应的中断源没有产生中断请求。SRCPND 在 **同一时刻** 可以有 **多个位同时被置 1**；
- INTMSK 中断屏蔽寄存器**：即使 SRCPND 某位已经置 1，但是如果此时 INTMSK 的对应位是 1（屏蔽），则 ARM 也不会响应中断。
- INTPND 中断未决寄存器**：在同一时刻 **只能有 1 个位被置 1**，表示 ARM 即将或已经在对该位相应的中断进行处理。
- ARM 响应中断的条件**：
 - （1）**允许中断**：CPSR 的 I=0（或 F=0）
 - （2）**有中断请求**：SRCPND 的对应位是 1（对子中断源，还必须 SUBSRCPND 对应位是 1）。
 - （3）**中断请求未被屏蔽**：INTMSK 的对应位是 0。
 - （4）**优先级最高**：INTPND 的对应位是 1。

5.3.3 中断编程实例

- 例 5-4**：如图所示，EINT0 接 S3C2440 的 GPFO，EINT2 接 S3C2440 的 GPF2，利用键盘 K1、K2 作为外部中断 EINT0、EINT2 的输入信号。键盘 K1、K2 按下，外部中断 EINT0、EINT2 被触发。



中断处理函数

```
static void_irq Eint0_ISR(void)           // 声明一个 Eint0_ISR 中断处理函数
{
    Delay(10);
    ClearPending(BIT_EINT0);             // 清零 SRCPND、INTPND 相应位
    Uart_Printf("EINT0 is occurred.\n");   // 输出提示：EINT0 发生
}
```

```
static void_irq Eint2_ISR(void)           // 声明一个 Eint2_ISR 中断处理函数
{
    Delay(10);
    ClearPending(BIT_EINT2);             // 清零 SRCPND、INTPND 相应位
    Uart_Printf("EINT2 is occurred.\n");   // 输出提示：EINT2 发生
}
```

说明：irq 是中断处理函数声明，使用此关键字声明的函数可以被编译器识别为中断处理函数。编译后的代码在处理异常事件前保存现场信息，处理异常事件后对现场信息进行恢复。

```
#define ClearPending(bit)                // 该函数在 2440addr.h 头文件里，此处仅用于说明
{
    rSRCPND = bit; // 向相应位置写 1，清除中断源未决寄存器 SRCPND
    rINTPND = bit; // 向相应位置写 1，中断未决寄存器 INTPND
}
```

例 5-4：GPF 外部中断触发方式配置

- 配置 EXTINT0（地址 0x56000088）：配置 EINT0、EINT2 都是下降沿触发方式。
- 01x → EINT0，01x → EINT2
EXTINT0 寄存器值：EINT7-EINT0 ...*01* **** *01*（* 表示不变）
- 写入寄存器：
rEXTINT0 = rEXTINT0 & 0xFFFF F9F9; // 0xFFFF F9F9=...1001 1111 1001
即 rEXTINT0 的 bit[2:1]=bit[10:9]=0，其他 bit 不受影响。
rEXTINT0 = rEXTINT0 | 0x0000 0202; // 0x0000 F2F2=...0010 0000 0010

EXTINT	位	描述	初始值
EINT3	[14:12]	确定 EINT3 信号有效方式。 000=低电平；001=高电平；01x=下降沿；10x=上升沿；11x=边沿。	000
EINT2	[10:8]	确定 EINT2 信号有效方式。 000=低电平；001=高电平；01x=下降沿；10x=上升沿；11x=边沿。	000
EINT1	[6:4]	确定 EINT1 信号有效方式。 000=低电平；001=高电平；01x=下降沿；10x=上升沿；11x=边沿。	000
EINT0	[2:0]	确定 EINT0 信号有效方式。 000=低电平；001=高电平；01x=下降沿；10x=上升沿；11x=边沿。	000

C 语言主程序

```
int Main()
{
    Uart_Init(115200); // 该函数在 2440addr.h 头文件里，作用是初始化
                        // Uart0 口，其中 Uart0 口的波特率设置为
                        // 115200bps
    Eint_Init();        // 该函数完成中断处理的初始化操作
    Enable_Eint();       // 该函数完成开外部中断的操作
    while (1)           // 死循环，等待中断事件：键盘 K1、K2 被按下，
                        // 外部中断 EINT0、EINT2 被触发
    {
        Uart_Printf("the main is running\n");
        Delay(50);
    }
}
```

例 5-4：GPF 引脚功能配置

- 配置 GPFCON（地址 0x56000050）：本例 GPF0 配置为 EINT0，GPF2 配置为 EINT2，GPF 端口其他引脚功能不变。10 → GPF0，10 → GPF2
- GPFCON 寄存器值：GPF7-GPF0 = **** *10* 10（* 表示不变）

写入寄存器：

- rGPFCON = rGPFCON & 0xFFCC; // 0xFFCC=1111 1111 1100 1100;
- rGPFCON = rGPFCON | 0x0022; // 0x0022=0000 0000 0010 0010;

即，GPFCON 的 bit[1:0]=10K1

GPFCON	位	描述
GPF7	[15:14]	00=输入；01=输出；10=EINT[7]；11=保留。
GPF6	[13:12]	00=输入；01=输出；10=EINT[6]；11=保留。
GPF5	[11:10]	00=输入；01=输出；10=EINT[5]；11=保留。
GPF4	[9:8]	00=输入；01=输出；10=EINT[4]；11=保留。
GPF3	[7:6]	00=输入；01=输出；10=EINT[3]；11=保留。
GPF2	[5:4]	00=输入；01=输出；10=EINT[2]；11=保留。

中断初始化函数

```
#define rGPFCON (*(volatile unsigned *)0x56000050) //GPF 的控制寄存器
#define rGPFUP (*(volatile unsigned *)0x56000058) //GPF 的上拉电阻寄存器
#define rEXTINT0 (*(volatile unsigned *)0x56000088) //GPF 的外部中断控制寄存器 0
#define pISR_EINT0 (*(unsigned *)(_ISR_STARTADDRESS+0x20)) //EINT0 中断向量地址
#define pISR_EINT2 (*(unsigned *)(_ISR_STARTADDRESS+0x28)) //EINT2 中断向量地址
void Eint_Init(void)
{
    rGPFCON = rGPFCON & 0xFFCC;
    rGPFCON = rGPFCON | 0x0022; //GPF0 设置为 EINT0,GPF2 设置为 EINT2
    rGPFUP=(1<<0); //GPF0 不使用上拉电阻
    rGPFUP=(1<<2); //GPF2 不使用上拉电阻
    rEXTINT0 = rEXTINT0 & 0xFFFF F9F9;
    rEXTINT0 = rEXTINT0 | 0x0000 0202; //配置 EINT0、EINT2 都是下降沿触发方式
    pISR_EINT0 = (unsigned)Eint0_ISR; //EINT0 产生请求时，调用处理
    pISR_EINT2 = (unsigned)Eint2_ISR; //EINT2 产生请求时，调用处理
}
```

5.4 DMA

- 5.4.1 S3C2440 芯片的 DMA 方式
- DMA 控制方式：主要用于高速 I/O 设备与存储器之间进行大批量数据传送。DMA 控制器控制数据直接在 I/O 设备与存储器之间传送，而不必经过微处理器内部的寄存器，这样就大大提高了数据传送的效率。
- S3C2440 的 DMA 系统支持 4 个独立通道的控制器。每个通道的 DMA 控制器可以在芯片内部高性能总线上的设备或外设总线上的设备、或分别在内部高性能总线和外设总线上的设备之间进行数据的无限次传送。
- DMA 优点：传输数据不受 CPU 的干涉。DMA 操作可以通过软件来启动，也可以通过内部设备的请求、或外部请求引脚的请求信号来启动。

例 5-4：GPF 引脚上拉电阻配置

- 配置 GPFUP（地址 0x56000058）：EINT0（GPF0）、EINT2（GPF2）作为外部中断，连接到 GPF 端口不需要上拉电阻。所以 GPF0、GPF2 不配置上拉电阻，其他位配置不变。
- 1 → GPF0，1 → GPF2
GPFUP 寄存器值：GPF7-GPF0 **** *1* 1（* 表示不变）
- 写入寄存器：
rGPFUP=(1<<0); // (1<<0) 将 1 左移 0 位 =0000 0001，按位“或”后赋值
rGPFUP=(1<<2); // (1<<2) 将 1 左移 2 位 =0000 0100，按位“或”后赋值
即 rGPFUP 的 bit[0]=bit[2]=1，其他 bit 不受影响。

GPFUP	位	描述
GPF7:0	[7:0]	1=对应的 GPIO 引脚上拉电阻不使能； 0=对应的 GPIO 引脚上拉电阻使能。

开中断函数

```
void Enable_Eint(void)
{
    rPRIORITY = 0x0000 007F; // 使用默认的循环优先级
    rINTMOD = 0x0000 0000; // 所有中断均为默认的 IRQ 中断
    rSRCPND |= BIT_EINT0|BIT_EINT2; // 写 1 SRCPND 的 EINT0、EINT2 位，清除未决位
    rINTPND |= BIT_EINT0|BIT_EINT2; // 写 1 INTPND 的 EINT0、EINT2 位，清除未决位
    rINTMSK = ~(BIT_EINT0|BIT_EINT2); // 清 0 INTMSK 的 EINT0、EINT2 位，允许中断
}
```

1. DMA 请求方式与请求源

- DMA 请求方式：2 种。
 - （1）硬件请求：H/W 请求模式，由选择的 DMA 请求源触发 DMA 操作。
 - （2）软件请求：S/W 请求模式，通过软件设置请求 DMA 操作。
- DMA 请求源：在 H/W 请求模式（硬件请求）有效，每个 DMA 通道有 7 个 DMA 请求源，可以选择其中的一个。
- 说明：虽然每个 DMA 通道有 7 个 DMA 请求源，但各不相同。

DMA 请求源

通道	请求源 0	请求源 1	请求源 2	请求源 3	请求源 4	请求源 5	请求源 6
Ch-0	nXDREQ0	UART0	SDI	Timer	USB device EP1	I2SSDO	PCMIN
Ch-1	nXDREQ1	UART1	I2SSDI	SPI0	USB device EP2	PCMOUT	SDI
Ch-2	I2SSDO	I2SSDI	SDI	Timer	USB device EP3	PCMIN	MICIN
Ch-3	UART2	SDI	SPI1	Timer	USB device EP4	MICIN	PCMOUT

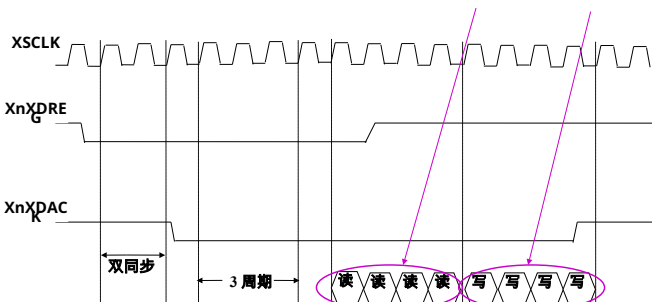
注：nXDREQ0 和 nXDREQ1 代表两个外部源（外部设备），I2SSDO 和 I2SSDI 分别代表 IIS 的发送和接收。

3. DMA 工作模式

- DMA 工作模式：2 种。
 - （1）**单服务模式**：单数据传送，一次 DMA 请求完成一个数据单元的 DMA 传送。
 - （2）**全服务模式**：连续数据传送，一次 DMA 请求完成一批数据单元的 DMA 传送。
- DMA 传输数据单元：一次 DMA 操作传送的数据宽度，包括 3 种：**字节、半字、字**。
- DMA 传输增量方式：一次 DMA 传送后，源、目的地址的变化情况，2 种：
 - （1）**增加**：在传输后地址增加（地址根据传输模式中数据大小的不同情况而增加）。
 - （2）**固定**：在传输后地址不改变（突发模式中，地址只在突发传输期间增加，但在传输后又回到其第一个值）。
- 外部 DMA 请求 / 应答协议：3 种。
 - （1）单服务请求模式。
 - （2）单服务握手模式。
 - （3）全服务握手模式。

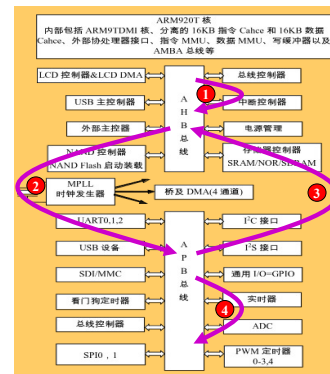
6. DMA 基本传输模式

- DMA 基本传输模式：在一个基本 DMA 传输操作中，根据传输的数据大小，分 2 种。
 - （1）**单次传输**：一个基本的 DMA 传输操作完成一次读和一次写。
 - （2）**突发传输**：一个基本的 DMA 传输操作完成四次连续读和四次连续写。



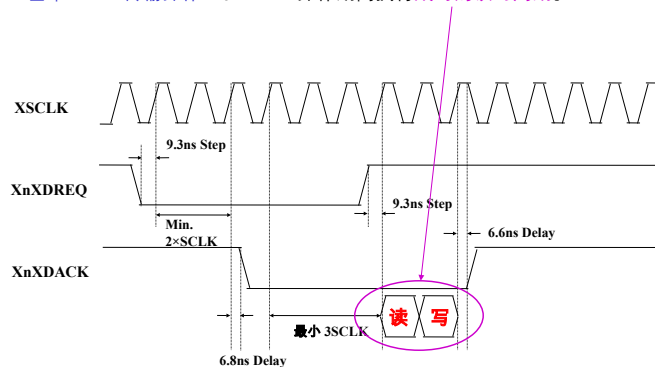
DMA 传输方向

- DMA 传输方向：4 种。
- 在“源→目的”之间实现“高性能总线（AHB）←→外设总线（APB）”的传送，情况如下：
 - （1）源在高性能总线（AHB）→目的在高性能总线（AHB）
 - （2）源在高性能总线（AHB）→目的在外设总线（APB）
 - （3）源在外设总线（APB）→目的在高性能总线（AHB）
 - （4）源在外设总线（APB）→目的在外设总线（APB）



4. 基本的 DMA 时序

- 基本 DMA 传输操作：在 DMA 操作期间执行成对的读写周期。



5.4.2 S3C2440 芯片的 DMA 寄存器

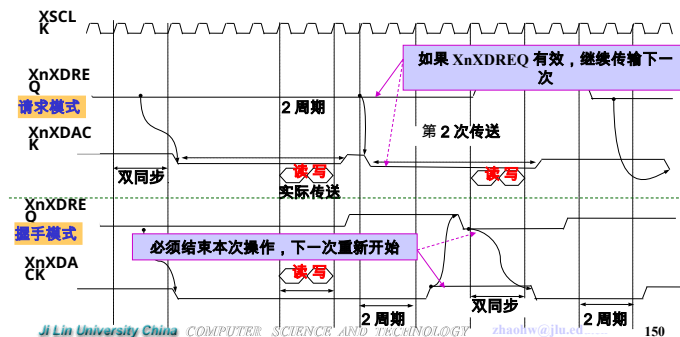
- 每个 DMA 通道有 9 个寄存器：6 个控制寄存器（控制 DMA 的传输），3 个状态寄存器（反映 DMA 的状态）。
- S3C2440 有 4 个 DMA 通道。4 个 DMA 通道共计有 36 寄存器。
- 6 个 DMA 控制传输的寄存器，可以进行读 / 写：
 - （1）DMA 初始源寄存器 DISRCn（DMA INITIAL SOURCE）
 - （2）DMA 初始源控制寄存器 DISRCCn（DMA INITIAL SOURCE CONTROL）
 - （3）DMA 初始目的寄存器 DIDSTn（DMA INITIAL DESTINATION）
 - （4）DMA 初始目的控制寄存器 DIDSTCn（DMA INITIAL DESTINATION CONTROL）
 - （5）DMA 控制寄存器 DCONn（DMA CONTROL）
 - （6）DMA 屏蔽触发寄存器 DMASKTRIGn（DMA MASK TRIGGER）
- 3 个状态监视的寄存器，只能进行读：
 - （1）DMA 状态寄存器 DSTATn（DMA STATUS）
 - （2）DMA 当前源寄存器 DCSRCn（DMA CURRENT SOURCE）
 - （3）DMA 当前目的寄存器 DCDSTn（CURRENT DESTINATION）

2. DMA 操作状态 - FSM 操作

- S3C2440 DMA 采用三态 FSM（有限状态机，Finite State Machine）操作：
 - （1）**状态 1：等待 DMA 请求**（初始状态）。一旦有请求，则进入状态 2。初始状态下，DMA ACK 信号（DMA 应答信号）和 INT REQ 信号（中断请求信号）都为 0。
 - （2）**状态 2：计数器加载**。在此状态下，DMA ACK 信号变成 1，计数器（CURR_TC）从 DCON 控制寄存器的 [19:0] 位加载计数初值。
 - （3）**状态 3：数据传输**。在此状态下，进行 DMA 基本操作。从源地址读取数据并写入目的地址。
- DMA 计数器：20 位，减 1 型计数器，每次当前传输结束后计数器的值减 1。计数器值减到 0 时，表示 DMA 操作结束。

5. DMA 传输模式

- DMA 传输模式：2 种。
 - （1）**请求模式**：也叫询问模式、查询模式，一个 DMA 数据单元传输结束后，如果 XnXDREQ 信号保持有效，就立刻进行下一次的 DMA 传输。
 - （2）**握手模式**：一个 DMA 数据单元传输结束后，结束本次 DMA 操作。若还想传输下一个数据，需要重新进行 DMA 请求。



1. 传输控制寄存器

- DMA 传输控制寄存器：6 个，主要用来控制 DMA 传输时的源地址起始值、目的地址起始值、数据块长度、DMA 模式等信息。
- （1）DMA 初始源寄存器
- DISRCn 功能：存放要传输的源数据起始地址。

寄存器	地址	读 / 写	描述	复位值
DISRC0	0x4B000000	读 / 写	DMA0 初始源地址寄存器	0x00000000
DISRC1	0x4B000040		DMA1 初始源地址寄存器	
DISRC2	0x4B000080		DMA2 初始源地址寄存器	
DISRC3	0x4B0000C0		DMA3 初始源地址寄存器	

DISRCn 寄存器格式

DISRC	位	描述	初始值
S_ADDR	[30:0]	用于传输的源数据基址（开始地址）。如果 CURR_SRC 为 0 且 DMA ACK 为 1，该位值将被装载到 CURR_SRC。	0

(3) DMA 初始目的寄存器 DIDSTn

- DIDSTn 功能：**存放要传输的**目的数据起始地址**。

寄存器	地 址	读 / 写	描 述	复 位 值
DIDST0	0x4B000008	读 / 写	DMA0 初始目标地址寄存器	0x00000000
DIDST1	0x4B000048		DMA1 初始目标地址寄存器	
DIDST2	0x4B000088		DMA2 初始目标地址寄存器	
DIDST3	0x4B0000C8		DMA3 初始目标地址寄存器	

DIDSTCn 寄存器格式

DIDSTC	位	描述	初始值
CHK_INT	[2]	当自动重载被设置，选择中断出现时间。 0= 若 TC 为 0，中断出现；1= 自动重载被执行后，中断出现。	0
LOC	[1]	用于选择 DMA 目的的位置。 0=DMA 目的在 高性能总线 (AHB) 上。 1=DMA 目的在 外设总线 (APB) 上。	0
INC	[0]	用于选择目的地址是否增加。0= 增加；1= 固定。 如果该位为 0，每次传输以后，地址增加 1（依据数据宽度）；如果该位为 1，每次传输以后，地址保持不变。	0

(2) DMA 初始源控制寄存器

DISRCn

- DISRC 功能：**选择源数据在高性能总线（AHB）上，还是在外部总线（APB）上，以及源地址的增量方式。

寄存器	地 址	读 / 写	描 述	复 位 值
DISRC0	0x4B000004	读 / 写	DMA0 初始源控制寄存器	0x00000000
DISRC1	0x4B000044		DMA1 初始源控制寄存器	
DISRC2	0x4B000084		DMA2 初始源控制寄存器	
DISRC3	0x4B0000C4		DMA3 初始源控制寄存器	

DIDSTn 寄存器格式

DIDST	位	描述	初始值
D_ADDR	[30:0]	要传输的目的数据基本地址（开始地址）。当且仅当 CURR_DST 为 0 并且 DMA ACK 为 1 时将此位的值锁存到 CURR_DST 中	0

(5) DMA 控制寄存器 DCONn

- DCONn 功能：**选择 DMA 传输模式（请求模式、握手模式）、DMA 基本传输模式（单次传输、突发传输）、DMA 工作模式（单服务模式、全服务模式）、DMA 通道请求源、DMA 请求方式（硬件请求、软件请求）、DMA 传输数据单元（字节、半字、字），设置 DMA 计数值。

寄存器	地 址	读 / 写	描 述	复 位 值
DCON0	0x4B000010	读 / 写	DMA0 控制寄存器	0x00000000
DCON1	0x4B000050		DMA1 控制寄存器	
DCON2	0x4B000090		DMA2 控制寄存器	
DCON3	0x4B0000D0		DMA3 控制寄存器	

DISRCn 寄存器格式

DISRC	位	描述	初始值
LOC	[1]	位 1 用来选择 DMA 源的位置。 0= 源在 高性能总线 (AHB) 上；1= 源在 外设总线 (APB) 上。	0
INC	[0]	位 0 被用于选择地址增加。0= 增加；1= 固定。 如果为 0，每次传输以后，地址增加相应的数据大小；如果为 1，每次传输以后，地址保持不变。	0

(4) DMA 初始目的控制寄存器 DIDSTCn

- DIDSTC 功能：**选择目的数据在**高性能总线 (AHB)** 上，还是在**外部总线 (APB)** 上，以及源地址的**增量方式**。

寄存器	地 址	读 / 写	描 述	复 位 值
DIDSTC0	0x4B00000C	读 / 写	DMA0 初始目标控制寄存器	0x00000000
DIDSTC1	0x4B00004C		DMA1 初始目标控制寄存器	
DIDSTC2	0x4B00008C		DMA2 初始目标控制寄存器	
DIDSTC3	0x4B0000CC		DMA3 初始目标控制寄存器	

DCONn 寄存器格式 1

DCON	位	描述	初始值
DMD_HS	[31]	用来选择 DMA 传输模式。 0= 选择 请求模式 ；1= 选择 握手模式 。	0
SYNC	[30]	用来选择 DREQ/DACK 同步信号。 0= DREQ / DACK 与 PCLK (APB 时钟) 同步。 1= DREQ / DACK 与 HCLK (AHB 时钟) 同步。	0
INT	[29]	用来使能 / 不使能 CURR_TC 产生中断。 0= 不使能；1= 使能。	0
TSZ	[28]	用来选择基本 DMA 传输的大小。 0= 执行单元传输；1= 执行阵发长度为 4 的 DMA 传输。	0
SERVMODE	[27]	用来选择服务模式。 0= 单独服务模式；1= 整体服务模式。	0
HWSRCSEL	[26:24]	各 DMA 通道请求源选择位。 通道 0：000：nXDREQ0；001：UART0；010：SDI；011：Timer 100：USB device EP1；101：HSSDO；110：PCMIN 通道 1：000：nXDREQ1；001：UART1；010：HSSDI；011：SPI 100：USB device EP2；101：PCMOUT；110：SDI 通道 2：000：HSSD0；001：HSSDI；010：SDI；011：Timer 100：USB device EP3；101：PCMIN；110：MICIN 通道 3：000：UART2；001：SDI；010：SPI；011：Timer	000
			162

SWHW_SEL	[23]	用来选择 S/W 还是 H/W 模式。 0=S/W 模式；1=H/W 模式。	0
RELOAD	[22]	设定重载开关选项 0=当前 DMA 传输完后，终点计数器自动重载； 1=当前 DMA 传输完后，DMA 通道被关闭。	0
DSZ	[21:20]	传输数据大小单位。 00=字节；01=半字；10=字；11=保留。	00
TC	[19:0]	初始 DMA 传输计数值。 传输的实际字节数由以下公式计算：DSZ x TSZ x TC。	0x00000

2. 状态寄存器

- 状态寄存器：记录 DMA 传输的状态，有 3 个寄存器，可以通过这些寄存器来了解 DMA 传输时的信息，以便于进行控制。
- (1) DMA 状态寄存器
- DMA 状态寄存器 (DSTATn) 共有 4 个：DSTAT0、DSTAT1、DSTAT2、DSTAT3，分别对应 4 个独立的 DMA 通道。
- 功能：DMA 就绪、忙状态，提供计数器值。
- 这 4 个寄存器地址分别为 0x4B000014、0x4B000054、0x4B000094、0x4B0000D4，复位后的初值为 0x00000000，且都是可读可写的。

DCSRCn 寄存器格式

DCSRC	位	描述	初始值
CURR_SRC	[30:0]	当前 DMA 通道的源地址值。	0x00000000

- DMASKTRIGn 功能：设置 DMA 通道的开启与关闭，设置软件 DMA 请求。

寄存器	地 址	读 / 写	描 述	复 位 值
DMASKTRIG0	0x4B000020	读 / 写	DMA0 屏蔽触发寄存器	0x00000000
DMASKTRIG1	0x4B000060		DMA1 屏蔽触发寄存器	
DMASKTRIG2	0x4B0000A0		DMA2 屏蔽触发寄存器	
DMASKTRIG3	0x4B0000E0		DMA3 屏蔽触发寄存器	

DSTATn 寄存器格式

DSTAT	位	描述	初始值
STAT	[21:20]	DMA 控制器的状态。 00=指示 DMA 控制器准备好接收下一个 DMA 请求； 01=指示 DMA 控制器忙。	0
CURR_TC	[19:0]	当前 DMA 计数器的值。 注：在每一次当前传输结束后该值减 1。	0x00000

(3) DMA 当前目的寄存器 DCDSTn

- DMA 当前目的寄存器 (DCDSTn) 共有 4 个：DCDST0、DCDST1、DCDST2、DCDST3，分别对应 4 个独立的 DMA 通道。
- 功能：提供当前 DMA 通道的目的地址值。
- 这 4 个寄存器地址分别为 0x4B00001C、0x4B00005C、0x4B00009C、0x4B0000DC，复位后的初值为 0x00000000，且都是可读可写的。

DMASKTRIG	位	描述	初始值
STOP	[2]	用来停止 DMA 操作。 0=正常；1=当一个基本操作完成后立即停止 DMA 操作。	0
ON_OFF	[1]	DMA 通道开关位。 0=关闭 DMA 通道；1=开启 DMA 通道。	0
SW_TRIG	[0]	用来在软件模式下触发 DMA 通道。 0=不触发；1=触发 DMA 操作。	0

(2) DMA 当前源寄存器 DCSRCn

- DMA 当前源寄存器 (DCSRCn) 共有 4 个：DCSRC0、DCSRC1、DCSRC2、DCSRC3，分别对应 4 个独立的 DMA 通道。
- 功能：提供当前 DMA 通道的源地址值。
- 这 4 个寄存器地址分别为 0x4B000018、0x4B000058、0x4B000098、0x4B0000D8，复位后的初值为 0x00000000，且都是可读可写的。

DCDSTn 寄存器格式

DCDST	位	描述	初始值
CURR_DST	[30:0]	当前 DMA 通道的目的地址值。	0x00000000